School of Computer Science
FACULTY OF ENGINEERING AND PHYSICAL SCIENCES

UNIVERSITY OF LEEDS

Final Report

DQN-Family Algorithms for Competitive Race Strategy in a Stochastic MARL Simulator

Branson Tay Liang Quan

Submitted in accordance with the requirements for the degree of BSc Computer Science

2025/26

COMP3931 Individual Project

The candidate confirms that the following have been submitted.

Items	Format	Recipient(s) and Date
Final Report	PDF file	Uploaded to Minerva (20/04/26)
Link to online code repository	URL	Sent to supervisor and assessor (25/04/26)

The candidate confirms that the work submitted is their own and the appropriate credit has been given where reference has been made to the work of others.

The candidate understands that failure to attribute material which is obtained from another source may be considered as plagiarism.

(Signature of Student) _____

© 2025/26 The University of Leeds and Branson Tay Liang Quan

Summary

This project investigates how incentive structure shapes emergent strategy when multiple reinforcement learning agents train together in a stochastic environment. The experimental domain is Formula 1 race strategy. The central variable is a reward-sharing coefficient that controls how much each agent values a teammate's performance alongside its own, ranging from purely self-interested to fully cooperative. By holding everything else constant and varying this coefficient, we isolate the effect of incentive design on the strategies agents develop.

Four variants of the Deep Q-Network family are compared as the base algorithm. In competitive settings, algorithm choice determines whether stable strategic behaviour emerges at all. Weaker variants collapse into passivity in close to half of all trials. The strongest variant, combining prioritised experience replay with double-target correction, avoids collapse entirely and becomes the stable foundation for all subsequent experiments.

Introducing cooperative reward into a purely competitive game produces a sharp breakdown rather than a gradual one. Below a certain threshold, partial alignment does not destabilise learning. Above it, agents converge to passivity in the majority of trials. Changing the game structure so that both agents can improve simultaneously resolves this. With a shared adversary to outperform together, cooperative incentive produces a large improvement in joint performance. An ablation study confirms the gain comes from the reward signal itself rather than from the change in game structure.

Scaling to more agents reverses the result. The cooperative advantage that appears in a three-agent setting disappears as team size grows, with the transition occurring abruptly at four agents. Curriculum training, which establishes individual policies before introducing cooperative reward, recovers much of the lost performance. This reveals two distinct failure modes. One arises from training order and can be resolved by scheduling. The other is a deeper credit assignment problem that requires architectural change beyond the independent learning framework.

A large language model given the race state as a natural language description with no prior training matches the best-trained agent across all tested conditions. The strategies are qualitatively different but arrive at similar outcomes, which suggests the environment is constrained enough for domain reasoning to approximate what the agent learned. A theoretically motivated reward formulation designed to credit each agent only for its own contribution fails both analytically and empirically. The approximation used inverts the cooperative gradient, penalising agents for their teammates' successes rather than rewarding their own actions.

The primary finding is that incentive design and agent count jointly determine whether cooperation produces advantage or harm, independently of algorithm choice. In small teams, careful alignment of incentives produces real cooperative gains. In larger teams, reward formula design alone is not sufficient and architectural changes become necessary.

Acknowledgements

I thank my friends and family who have been supportive of me through my journey of Computer Science. I also thank my supervisor, Brandon Bennett for providing guidance where needed.

The automated hyperparameter search that determined the shared training configuration for this project used Andrej Karpathy's Autoresearch tool (Karpathy, 2024), available at `https://github.com/karpathy/autoresearch`.

Contents

1 Introduction and Background Research **1**
1.1 Introduction . . . 1
1.2 What the research has established . . . 1
1.2.1 Motorsport simulation . . . 1
1.2.2 Reinforcement learning in competitive environments . . . 2
1.2.3 Multi-agent incentive design . . . 2
1.3 The research gap and this project's contribution . . . 3
1.4 DQN substrate as the measurement apparatus . . . 4
1.5 Aim, objectives, and research questions . . . 5
1.5.1 Aim . . . 5
1.5.2 Research questions and hypotheses . . . 6
1.5.3 Objectives . . . 6

2 Methodology **7**
2.1 Methodology overview . . . 7
2.2 Simulator design . . . 7
2.2.1 Overtake success probability . . . 7
2.2.2 Config-driven action and feedback contracts . . . 8
2.2.3 Finish-first reward contract . . . 8
2.2.4 Design parameter choices . . . 9
2.3 Experimental methodology . . . 10
2.3.1 Phase-to-RQ mapping . . . 10
2.3.2 Phase validation and promotion criteria . . . 11
2.3.3 Measurement plan . . . 11
2.3.4 Phase 0: System integrity validation . . . 12
2.3.5 Phase 1: Single-agent smoke validation . . . 12
2.3.6 Phase 2: Single-agent DQN-family baseline . . . 12
2.3.7 Phase 3: Independent learner MARL . . . 12
2.3.8 Phase 4: Alpha (α) reward mixing . . . 12
2.3.9 Phase 5: Non-zero-sum game design . . . 13
2.3.10 Phases 6 and 7: Multi-team scaling . . . 13
2.3.11 Phase 8: Curriculum Alpha (α) scheduling . . . 14
2.3.12 Phase 9: LLM semantic baseline . . . 14
2.3.13 Phase 10: Difference rewards . . . 14
2.4 Project management . . . 15
2.4.1 Sprint structure and deliverable mapping . . . 15
2.4.2 Version control and gate discipline . . . 15
2.4.3 Research question independence . . . 15

3 Implementation and Validation **16**
3.1 System architecture . . . 16
3.1.1 Module overview . . . 16
3.1.2 Config-driven design as a fairness control . . . 17
3.2 Agent implementations . . . 17
3.2.1 DQN-family agents . . . 17
3.2.2 Base agent . . . 18
3.2.3 Shared training infrastructure . . . 18

3.3 Benchmark infrastructure . . . 18
3.3.1 MARL evaluation runner . . . 19
3.3.2 MARL and phase extensions . . . 19
3.3.3 Challenges and resolutions . . . 19
3.4 Gate validation summary . . . 20

4 Results, Evaluation and Discussion **21**
4.1 Preliminary validation . . . 21
4.2 RQ1: What strategic behaviours emerge and how robust are they? . . . 21
4.3 RQ2: How does incentive structure alter strategy? . . . 23
4.4 RQ3: Does shared incentive produce genuine cooperative advantage at scale? . . . 26
4.5 RQ4: Do alternative cooperative reward formulations preserve the advantage? . . . 28
4.6 Validity check: LLM semantic baseline . . . 29
4.7 Discussion . . . 29
4.7.1 Synthesis . . . 29
4.7.2 Validity threats and limitations . . . 30
4.7.3 Future directions . . . 30

References **31**

Appendices **35**

A Self-Appraisal **35**
A.1 Critical self-evaluation . . . 35
A.2 Personal reflection and lessons learned . . . 36
A.3 Legal, social, ethical and professional issues . . . 37
A.3.1 Legal issues . . . 37
A.3.2 Social issues . . . 37
A.3.3 Ethical issues . . . 38
A.3.4 Professional issues . . . 38

B External Material **39**
B.1 Overtaking zone calibration figures . . . 39

C Phase 0 and Phase 1 Experimental Validation **41**
C.1 Phase 0: System Integrity . . . 41
C.1.1 Objective and scope . . . 41
C.1.2 Test inventory . . . 41
C.1.3 Defects found and corrected . . . 41
C.1.4 Phase 0 gate outcome . . . 42
C.2 Phase 1: Single-Agent Smoke Evaluation . . . 42
C.2.1 Objective and configuration . . . 42
C.2.2 s0 results: three training seeds . . . 42
C.2.3 Risk behaviour at s0 . . . 43
C.2.4 Stochasticity degradation: s0, s1, and s2 . . . 43
C.3 Phase 1 findings and implications . . . 43
C.3.1 Win rate floor: La Source structural dominance . . . 43
C.3.2 AGGRESSIVE flip as a DQN overestimation indicator . . . 44
C.3.3 Zone discrimination collapse at s2 . . . 44
C.3.4 Seed variance as a robustness indicator . . . 44
C.3.5 Phase 1 gate summary . . . 45

D DQN variant formulations and training specifics **46**

D.1 Variant TD targets . . . 46
D.2 Network and replay specifications . . . 47
D.3 Exploration and optimisation schedule . . . 47
D.4 Hyperparameter tuning protocol . . . 47

E DQN Training Loop **48**

F System Class Diagram **50**
F.1 Per-module responsibilities . . . 50
F.2 UML class diagram . . . 50

G Simulator specification details **52**
G.1 Stochasticity levels . . . 52
G.2 Base Agent specification . . . 52
G.3 Run reproducibility through telemetry . . . 52

H Benchmark infrastructure specification **54**
H.1 Single-trial runner . . . 54
H.2 Benchmark matrix orchestrator . . . 54
H.3 Telemetry schema . . . 54

I Phase 3 crossplay bias arithmetic **55**

J Phase 4 secondary patterns **56**
J.1 Non-monotonic zone convention formation . . . 56
J.2 Stochasticity role inversion . . . 56

K MARL and phase extensions **57**
K.1 Phase 3: Concurrent learners . . . 57
K.2 Phases 4 to 10: Reward, team, curriculum, and agent extensions . . . 57

Chapter 1

Introduction and Background Research

1.1 Introduction

In multi-agent reinforcement learning (MARL), agents that learn alongside each other do not just solve a task. They adapt to one another. Learning stability and emergent strategy both depend on how incentives are structured between agents, and that dependency is not well characterised. Much of MARL has historically focused on fully cooperative or fully competitive extremes, with many methods tailored to those cases. The partially-aligned or general-sum regime is an emerging research area that remains less mature.

The question this project investigates is whether shared incentive between concurrently learning agents produces genuine cooperative advantage, or merely redistributes competitive outcomes. When two agents race head-to-head, one agent's gain is the other's loss by definition, so no reward formula can produce net joint improvement. Introducing a common adversary lifts that constraint, and both agents can then improve simultaneously by outperforming the third party. Whether shared incentive actually exploits that opening, and how far it can be pushed before learning destabilises, is underexplored in existing literature.

Formula 1 overtaking zone decisions are a well-suited environment to study this. Decisions are discrete and repeated, outcomes are probabilistic given the same zone and gap, and the opponent adapts across laps. The game structure is also adjustable. The same simulator, agents, and reward formula can be tested with or without a common adversary, isolating game structure as an independent variable.

To answer this, the project builds an F1-inspired simulator and trains reinforcement learning (RL) agents across zero-sum and non-zero-sum game structures, with the reward-sharing coefficient α varied from pure competition to pure cooperation. The work draws on three fields, namely motorsport simulation, RL in competitive environments, and multi-agent incentive design.

1.2 What the research has established

A structured comparison of related studies is in Appendix Table B.2.

1.2.1 Motorsport simulation

The earliest motorsport strategy research treats the simulator as infrastructure for strategy evaluation rather than algorithm comparison. A lap-wise simulation was built modelling tyre degradation, fuel effects, pit stops, and overtaking processes from publicly accessible race data

(Heilmeier et al., 2018). The goal was to find a good strategy for a given race, which is a different problem from comparing how learning algorithms develop strategies under controlled conditions. This work was extended with neural network surrogates showing that data-driven approximations can accelerate strategy exploration (Heilmeier et al., 2020), and online planning for Formula 1 confirmed that planning methods can produce competitive decisions in dynamic conditions (Piccinotti et al., 2021).

The Formula E literature narrows the gap. Multi-car extensions of RL-based race strategy showed that multiple concurrently learning agents can produce competitive results in motorsport (Liu et al., 2024). Learning-based multi-agent race strategies in Formula 1 have also been reported (Fieni et al., 2026). Across this body of work, no study has systematically varied the incentive structure between concurrent learning agents, or tested how reward alignment changes the strategies that emerge.

1.2.2 Reinforcement learning in competitive environments

Reinforcement learning frames decision-making as an agent that observes a state, selects an action, and receives a reward. Across repeated episodes the agent adjusts its policy, the mapping from states to actions, to maximise long-term reward. Value-based methods estimate a Q-value for each action in each state, and select the action with the highest Q-value at each step (Mnih et al., 2015).

Applied to competitive settings, the framework has produced strong results over the past decade, from board games (Silver et al., 2018) to real-time strategy at scale (Berner et al., 2019; Vinyals et al., 2019). These systems span three substrate families. Value-based methods pick the action with the highest estimated Q-value and suit small discrete action spaces. Policy-based methods learn a policy directly and scale to continuous control. Actor-critic combines both and has become the default for large-scale domains. Discrete decisions with a modest compute budget suit value-based learning. Within that family, the Deep Q-Network (DQN) was the first to scale value-based learning to high-dimensional state spaces using deep neural networks (Mnih et al., 2015). Section 1.4 returns to DQN with the specific requirements of this project in view and selects a variant for the experimental matrix.

The central challenge in competitive RL is that a policy learned against a fixed opponent degrades when that opponent adapts. Self-play addresses this by training an agent against increasingly skilled versions of itself, though it introduces failure modes including cyclic policy dynamics and exploitability by novel opponents. Self-play is a single-agent method adapted to adversarial settings, and the setting where multiple agents train concurrently is not its target.

1.2.3 Multi-agent incentive design

In multi-agent settings, outcomes depend on joint actions rather than individual ones, and agents observe only local state (Buşoniu et al., 2008). Because all agents update at the same time, the environment any one agent experiences keeps shifting as the others' policies change. This effect,

called non-stationarity, is the central technical challenge in multi-agent learning (Hernandez-Leal et al., 2019).

Mixed-motive settings, where agents' interests are partially aligned and partially conflicting, remain less developed than the pure cooperative and zero-sum cases in both benchmarking and realistic domains (Du et al., 2023). In motorsport, this space is only recently emerging (Fieni et al., 2026).

Independent learners are the natural first baseline. Each agent stores past experience in a replay buffer and samples from it to update its own policy. When the opponent's policy changes, stored experience from an earlier version of that opponent misdirects updates, and learning can become unstable (Foerster et al., 2017; Matignon et al., 2012). Both non-stationarity and the difficulty of assigning credit for shared outcomes worsen as more agents are added (Agogino and Tumer, 2004; Zhao et al., 2025).

Credit assignment is the problem of distributing a shared team reward among agents in proportion to their individual contribution, and it remains unsolved in team reward settings (Panait and Luke, 2005; Ning and Xie, 2024). Two broad families of solutions exist at the reward-signal level. The first mixes individual and team rewards to balance credit assignment against cooperative alignment (Wang et al., 2022), which is the mechanism the α coefficient implements. The second gives each agent a reward based on how the team would have performed in its absence. This counterfactual subtraction isolates an agent's individual contribution, and difference rewards are the canonical form (Wolpert and Tumer, 2001; Devlin et al., 2014). Exact computation is impractical at scale, so the team's observed mean is often substituted as a stand-in for the counterfactual (Yang et al., 2018). The two are not the same quantity, and conflating them can lose the alignment property that difference rewards rely on (Devlin et al., 2014).

Two failure modes arise in this space. Training-order collapse happens when cooperative reward is introduced before individual policies have stabilised, overwriting competent baselines with passive behaviour (Matignon et al., 2012). Credit-assignment failure happens when overlapping individual contributions prevent team reward from producing useful per-agent gradients (Panait and Luke, 2005; Rashid et al., 2018). Telling them apart in practice requires varying incentive alignment across training rather than holding it fixed. An alternative route to the same credit-assignment problem is at the level of training setup rather than reward signal, through centralised training with decentralised execution (CTDE) (Rashid et al., 2018).

The field has mapped each piece in isolation. How they interact under independent concurrent learning, across reward-signal design, agent count, and game structure, remains less well understood.

1.3 The research gap and this project's contribution

Multi-agent learning has been studied most extensively at the two ends of the incentive spectrum. Fully cooperative settings have produced a mature literature on credit assignment and centralised training. Fully competitive settings, especially two-player zero-sum games, have produced strong

methods around opponent adaptation, self-play, and equilibrium-oriented learning. The mixed-motive regime between them is the space this project occupies, and studies in this regime typically isolate one factor at a time. What is less well understood is the joint behaviour of reward-signal design, agent count, and game structure when all three are varied together on the same learning substrate, with diagnostics that distinguish genuine cooperative advantage from redistribution of competitive outcomes.

This project builds the apparatus that can answer that joint question. The simulator is an F1-inspired race where game structure is switched by the presence or absence of a common adversary, and agent count is configurable from two to four DQN learners. A DQN-family substrate is selected to give stable single-agent learning before any multi-agent phase begins, so that algorithm-level differences do not confound the incentive-structure comparisons. The experimental programme sweeps the reward-sharing coefficient α across both game structures and the agent-count range, and evaluates alternative cooperative reward formulations against the α-mixed baseline. The output is an empirical characterisation of how reward sharing, game structure, and agent count jointly shape emergent strategy and learning stability.

1.4 DQN substrate as the measurement apparatus

The project's central measurement is whether changes in emergent strategy can be attributed to incentive structure, which requires controlling for algorithm-level differences (Henderson et al., 2018; Agarwal et al., 2021). The substrate needs variants that are small and well-characterised, a decision space that matches discrete action selection at a small number of zones, and off-policy learning so rare overtaking events can be revisited during training (Mnih et al., 2015; Schaul et al., 2016).

The DQN family has this structure by design. Each variant in the lineage was introduced to correct a named failure mode of the previous one. Double DQN targets Q-value overestimation under noisy rewards (van Hasselt et al., 2016). Dueling DQN separates state value from action advantage to stabilise learning where the choice of action has little effect on the outcome (Wang et al., 2016). Prioritised experience replay addresses uniform sampling when informative transitions are rare (Schaul et al., 2016). Rainbow combines these into a single algorithm (Hessel et al., 2018). Because each mechanism has a clear target, differences between variants map onto specific known biases rather than unrelated implementation choices. Independent Q-learning, the standard value-based MARL baseline (Rashid et al., 2018), places the family within the multi-agent literature.

The comparison set is therefore vanilla DQN, Double DQN, Dueling DQN, and a Rainbow-style variant (rainbow-lite) combining the three with prioritised replay. Chapter 2 sets out the experimental methodology, and Chapter 3 covers the implementation.

1.5 Aim, objectives, and research questions

1.5.1 Aim

The aim of this project is to characterise how cooperation between self-interested learning agents changes as incentive alignment, agent count, and reward formulation are varied within a single controlled simulator. Where cooperation fails or breaks down, the project identifies the underlying mechanism, distinguishing between training-order effects, credit-assignment limits at scale, and formulation-specific failures.

1.5.2 Research questions and hypotheses

RQ1. What strategic behaviour emerges when independent reinforcement learning agents train concurrently under pure competition?

H1. Concurrent training produces differentiated zone coverage and opposing risk profiles through iterated best-response dynamics. No explicit coordination mechanism is required.

RQ2. How does varying the reward-sharing coefficient α alter emergent strategy and learning stability in two-agent zero-sum and non-zero-sum settings, and can curriculum scheduling recover cooperative performance when fixed α fails?

H2. Above a threshold, sharing too much reward turns successful overtakes into a net penalty and both agents drift into passive play. In non-zero-sum races with a common rival, partial sharing helps up to that same threshold and then starts to hurt. Curriculum scheduling recovers some of the lost performance.

RQ3. How many agents can a non-zero-sum race hold before cooperation stops working, and does gradual scheduling change the answer?

H3. Beyond a certain agent count, no value of α and no training schedule brings cooperation back, because the problem is credit assignment rather than training order.

RQ4. Do alternative cooperative reward formulations preserve the advantage that α-based mixing produces?

H4. The mean-field substitute for the true counterfactual inverts the cooperative signal, so the advantage observed under α-based mixing will not reproduce under this approximation.

1.5.3 Objectives

- Build a racing simulator with configurable game structure, agent count, and diagnostic output that distinguishes cooperative advantage from redistributed competitive outcomes.
- Pick a DQN variant that trains stably in the single-agent setting to act as a controlled substrate for the comparisons that follow.
- Measure how the reward-sharing coefficient α affects emergent strategy and learning stability across zero-sum and non-zero-sum game structures.
- Test whether cooperative performance breaks down as agent count grows, and whether curriculum scheduling can recover it.
- Evaluate alternative cooperative reward formulations, and check whether a zero-shot language model with structured domain priors can match trained agents in the simulator's scope.

Chapter 2

Methodology

2.1 Methodology overview

The methodology combines a controlled apparatus that varies incentive alignment, agent count, and reward formulation within a single simulator, a measurement substrate that isolates incentive structure from algorithmic variance, and a phased experimental sequence with promotion gates so claims at later phases rest on verified foundations.

2.2 Simulator design

The simulator models a race as a sequence of discrete decision points at overtaking zones, rather than modelling lap-time physics at the second-by-second level (Heilmeier et al., 2018). This quasi-static approach keeps each race cheap enough to support multi-seed sweeps across α, game structure, and agent count.

Zone difficulties are grounded in real race data. Using the FastF1 library, per-driver position changes from the 2023 Belgian Grand Prix were plotted against track distance. Position changes cluster at specific points on the circuit, indicating where overtaking happens in practice (Appendix B.1). Because position changes are recorded at timing loops rather than exact on-track position, these values capture relative difficulty across zones rather than precise overtake points.

The analysis gives nine zones at Spa-Francorchamps. La Source (the hairpin at the start of the lap) has a difficulty of 0.2, reflecting how frequently overtakes happen there. The remaining zones sit between 0.7 and 0.9, consistent with how rare overtakes are at those locations in real racing. The full mapping from track location to zone is in Appendix B.2.

The simulator supports three stochasticity levels (s0, s1, s2) for robustness testing, since RL evaluation is sensitive to environmental variance (Henderson et al., 2018; Agarwal et al., 2021; Sankary and Ostfeld, 2018). s0 is the noise-free primary comparison track, while s1 and s2 introduce increasing outcome variance without altering zone difficulty ordering in expectation. Appendix G gives the full specification.

2.2.1 Overtake success probability

When an agent chooses to attempt an overtake, the simulator resolves the outcome by computing a success probability and sampling from it. The probability is computed as

$$P_{\text{success}} = \text{clip}(\, p_{\text{base}} + \delta_{\text{risk}} + \delta_{\text{gap}} + \epsilon,\ p_{\text{min}},\ p_{\text{max}})$$

where each term is defined as follows.

Base probability. The zone difficulty $d \in [0, 1]$ controls the base success rate. An easy zone with $d = 0.2$ yields $p_{\text{base}} = 0.8$, while a hard zone with $d = 0.9$ yields $p_{\text{base}} = 0.1$.

$$p_{\text{base}} = 1 - d$$

Risk adjustment. The agent selects from three risk levels, each adding a fixed offset to the probability.

$$\delta_{\text{risk}} = \begin{cases} -0.08 & \text{CONSERVATIVE} \\ 0.00 & \text{NORMAL} \\ +0.12 & \text{AGGRESSIVE} \end{cases}$$

A more aggressive attempt therefore shifts the probability upward but also incurs a larger failure penalty in the reward signal, creating a risk-reward trade-off for the agent to learn.

Gap modifier. A closer following distance increases the probability of a successful overtake the closer you are to the car in front. The gap g in kilometres is normalised against a reference distance of 0.1 km, giving

$$\delta_{\text{gap}} = \left(1 - \frac{g}{0.1} - 0.5\right) \times 0.3$$

Stochastic noise. Gaussian noise $\epsilon \sim \mathcal{N}(0, \sigma)$ is added to the probability before clipping, with σ set by the active stochasticity level (see Appendix G).

The final probability is clipped to $[0.02,\ 0.95]$ to avoid degenerate certainty in either direction.

2.2.2 Config-driven action and feedback contracts

The decision trigger conditions, action space, and observation vector are all defined in `config.json` and summarised in Table 2.1. Each (driver, zone, lap) combination may produce at most one decision per race, resolved when the driver reaches the zone. The state dimension is derived from config, so all DQN-family algorithms consume the same observation contract within a given run.

A conservative attempt reduces the success probability by 0.08 relative to normal, while an aggressive attempt raises it by 0.12. However, failure penalties in the reward signal scale in the opposite direction. A failed conservative attempt costs -0.5, a normal failure -1.0, and an aggressive failure -1.5. The optimal risk level therefore depends on zone difficulty, current gap, and positional context.

2.2.3 Finish-first reward contract

The reward signal combines four components.

Table 2.1: Action and feedback contract summary.

Decision trigger (all conditions must hold)	
Proximity	Driver within 0.2 km of an overtaking zone
Car ahead	A car exists within 0.1 km
Gap	Gap to that car is at most 0.1 km
Cooldown	Driver is not in post-attempt cooldown window
Action space (4 discrete actions)	
0	Hold position
1	Conservative overtake attempt
2	Normal overtake attempt
3	Aggressive overtake attempt
Observation vector (6 features)	
o_1	Distance to next overtaking zone (normalised)
o_2	Gap to nearest car ahead (normalised)
o_3	Zone difficulty
o_4	Binary indicator: car ahead exists
o_5	Current position (normalised)
o_6	Laps remaining (normalised)

$$R = R_{\text{outcome}} + R_{\text{persistent_position}} + R_{\text{tactical}} + R_{\text{penalty}}$$

The objective hierarchy prioritises the finishing position. Outcome reward is terminal and dominant (weight 2.0), rewarding start-to-finish position gain. Persistent-position reward provides dense shaping from position changes that survive decision resolution (weight 0.1). Tactical reward gives local feedback for overtake outcomes at the risk-scaled failure costs listed above (weight 0.05). Penalty reward is a defensive component for unfinished races at weight 2.0 and rarely fires in practice because races typically complete.

Each experiment runs over five laps at Spa-Francorchamps. Agent composition varies by phase, ranging from one DQN plus the Base Agent up to four DQN agents plus Base, with the exact configuration per phase given in Table 2.2.

The Base Agent is a fixed stochastic heuristic comparator. It samples a risk level from zone difficulty and gap information, then samples whether to attempt the pass from a risk-dependent and difficulty-dependent probability. The agent does not update over time, which gives DQN policies a stationary non-learning baseline to be evaluated against. The full specification is in Appendix G.

2.2.4 Design parameter choices

Several numerical values in the simulator specification (risk adjustments, failure penalties, reward weights, clipping bounds, stochasticity sigmas, gap modifier scaling, episode budgets, lap counts) are calibration choices made during early development. What is load-bearing for the methodology is the structural relationships between them, not the magnitudes. The risk-adjustment ordering, the failure-penalty inverse ordering, the reward weight hierarchy, and the s0 / s1 / s2 variance progression must all hold for the methodology to work. Specific values could differ without changing the conclusions. The structural design choices themselves (the zero-sum to non-zero-

sum game structure switch, the agent-count progression, the α sweep across $[0, 1]$, the curriculum warm-up then ramp then hold, and the Phase 10 mean-field approximation) are described in their respective phase subsections.

2.3 Experimental methodology

2.3.1 Phase-to-RQ mapping

Each phase in the experimental programme either answers a research question from Chapter 1, or establishes a prerequisite for phases that do. The rightmost column of Table 2.2 shows the mapping.

Phase 0 and Phase 1 are validation gates that confirm the simulator is correct and a learning signal exists before any experimental claim is made. Phase 2 selects the DQN substrate that all subsequent phases use, so that algorithm differences do not confound incentive-structure comparisons. Phase 3 onwards carry the research-question load.

Several phases use an ablation design to isolate which specific mechanism drives a given effect. An ablation holds everything constant across conditions except the mechanism under test, so that any outcome difference is attributable to that mechanism. Phases 3, 5, and 7B all use this pattern for their respective isolations, detailed in each phase's subsection.

Phase 9 sits outside the RQ structure because its question is not about cooperation dynamics but about whether the learning problem genuinely requires adaptive strategy. If a zero-shot Large-Language Model (LLM) matches a trained DQN agent, the basis for the MARL investigation is weakened, so Phase 9 is a validity check on the project as a whole.

Table 2.2: Experimental phase summary. All phases are evaluated across stochasticity levels s0, s1, and s2. Agent counts refer to DQN-family learners; the Base adversary is additionally present where noted.

Phase	**Agents**	**Game structure**	α	**Reward mode**	**Training**	**Question**
0	—	—	—	—	—	Validation
1	1 DQN + Base	Single-agent	—	Individual	Fixed	Validation
2	1 DQN + Base	Single-agent	—	Individual	Fixed	Substrate
3	2 DQN	Zero-sum	0.0	Individual	Fixed	RQ1
4	2 DQN	Zero-sum	0.0–1.0	α-mixed	Fixed	RQ2
5	2 DQN + Base	Non-zero-sum	0.0–1.0	α-mixed	Fixed	RQ2
6	4 DQN + Base	Non-zero-sum	Various	α-mixed	Fixed	RQ3
7A	3 DQN + Base	Non-zero-sum	Various	α-mixed	Fixed	RQ3
7B	4 DQN + Base	Non-zero-sum	Low	α-mixed	Fixed	RQ3
8A	3 DQN + Base	Non-zero-sum	0.75	α-mixed	Curriculum	RQ3
8B	4 DQN + Base	Non-zero-sum	0.75	α-mixed	Curriculum	RQ3
9	1 LLM + Base	Single-agent	—	Individual	None	Validity
10	3 DQN + Base	Non-zero-sum	0.75	Difference	Fixed	RQ4

2.3.2 Phase validation and promotion criteria

Each phase is validated against criteria appropriate to its scope before downstream phases proceed. Early phases carry heavier gates because they establish the foundations for everything that follows.

Three formal gates govern the early phases. Gate G0 is a hard automated integrity gate where all 12 simulator, reward, and telemetry tests pass or the gate fails (Appendix C). Gate G1 is a methodological smoke gate verifying that a positive learning signal exists before the experimental matrix is committed to. Gate G2 is a formal benchmark promotion gate that selects the DQN variant used for Phases 3 onwards.

Phase 3 and later phases carry phase-specific validation rather than uniform gates. Each phase has documented pass criteria tied to its experimental role, with the pass decision recorded alongside the phase analysis. A failed validation does not proceed. The phase is corrected and re-run under the same protocol. Validation is part pre-committed (pass criteria defined before the phase runs) and part investigation-driven (additional robustness checks or bias controls added in response to what the phase data shows). Both are reported alongside the phase results in Chapter 4.

2.3.3 Measurement plan

Each phase is evaluated at three stochasticity levels. s0 is the primary comparison track with noise removed. s1 and s2 are robustness tracks testing whether the s0 ranking survives under increasing outcome uncertainty. Within each level, multiple seeds estimate the variance of the learning process. A seed fixes the pseudo-random streams driving network initialisation, exploration, and replay sampling, so two runs at the same seed produce identical results. Different seeds produce different learning trajectories because RL is path-dependent, and multiple seeds therefore give a distribution of outcomes that is the basis for 95% confidence intervals (CI95) across trials (Henderson et al., 2018). Three seeds per evaluation cell is the design baseline, which is on the low end of current RL practice and widens confidence intervals relative to five- or ten-seed designs.

The core performance metrics are win rate, joint beat-base rate, per-agent beat-base rate, and positional advantage. Win rate is the proportion of evaluation races won by the relevant DQN agent, with 95% CIs over evaluation races. Joint beat-base rate is the proportion of races in which every active DQN agent finishes ahead of the Base adversary, and becomes the central cooperative advantage metric from Phase 5 onwards. Per-agent beat-base rate applies the same test individually, distinguishing joint improvement from one agent carrying the result. Positional advantage is the mean finishing-position gap between two DQN agents.

Strategy is characterised through zone differentiation and risk differentiation indices. Zone differentiation is the mean absolute difference in per-zone attempt rates between the two agents. Risk differentiation is the mean absolute difference in attempt-conditioned risk proportions (the fraction of attempts at each risk level, computed over attempted overtakes only). These test

RQ1's prediction of asymmetric role development.

2.3.4 Phase 0: System integrity validation

Phase 0 validates the simulator, reward accounting, stochasticity implementation, and telemetry contract before any learning claims are made. Test Group A checks simulator correctness, including position tracking, lap counting, and zone sequencing. Group B validates stochastic probability distributions using repeated sampling to confirm the noise term produces the expected spread at each level. Group C verifies the telemetry schema and reward component accounting.

2.3.5 Phase 1: Single-agent smoke validation

Phase 1 runs vanilla DQN under the smoke protocol (200 training episodes, three seed sets of 20 evaluation races per seed) to confirm that a positive learning signal exists before the full Phase 2 budget is committed. Three seeds are used at s0 and one seed each at s1 and s2. Gate criteria require win rates above 0.5 across all s0 seeds with the confidence interval lower bound also clearing 0.5, zero fairness violations, and interpretable degradation across stochasticity levels.

2.3.6 Phase 2: Single-agent DQN-family baseline

Phase 2 establishes the DQN-family reliability baseline under a controlled single-agent contract. Each variant (vanilla, double, dueling, rainbow-lite) is trained and evaluated against the fixed Base Agent with identical reward schema, observation contract, stochasticity level, episode budgets, and seeds (Henderson et al., 2018). Phase 2 produces the per-algorithm stochasticity profiles that inform substrate selection for all subsequent phases. The shared training configuration was determined by systematic hyperparameter tuning prior to Phase 2, so no variant is advantaged by an arbitrary configuration choice.

2.3.7 Phase 3: Independent learner MARL

Phase 3 replaces the fixed Base Agent with a second concurrent DQN learner. Both agents train at the same time, each with its own replay buffer and network parameters. This is the independent Q-learning (IQL) formulation, the standard value-based MARL baseline where each agent treats other agents as part of a non-stationary environment (Foerster et al., 2017; Hernandez-Leal et al., 2019). Phase 3 is gated on Phase 2 passing all promotion criteria.

2.3.8 Phase 4: Alpha (α) reward mixing

Phase 4 groups the two DQN agents into a single team and introduces a reward-sharing mechanism controlled by the coefficient α. Each agent's effective reward is a weighted combination of its own reward and its teammate's reward.

$$R_{\text{mixed}} = (1 - \alpha) \cdot R_{\text{own}} + \alpha \cdot R_{\text{teammate}}$$

At $\alpha = 0$ each agent receives only its own reward, reproducing the Phase 3 IQL setup. At $\alpha = 1$ each agent receives only its teammate's reward. Intermediate values produce partial alignment. The team structure carries through every subsequent phase.

Five α values are tested in Phase 4 (0.0, 0.25, 0.50, 0.75, and 1.0) to identify whether there is a threshold beyond which cooperative incentive causes learning to collapse rather than cooperate. This sweep distinguishes a gradual performance degradation at high α from a structural passivity failure, which is the core empirical question of RQ2.

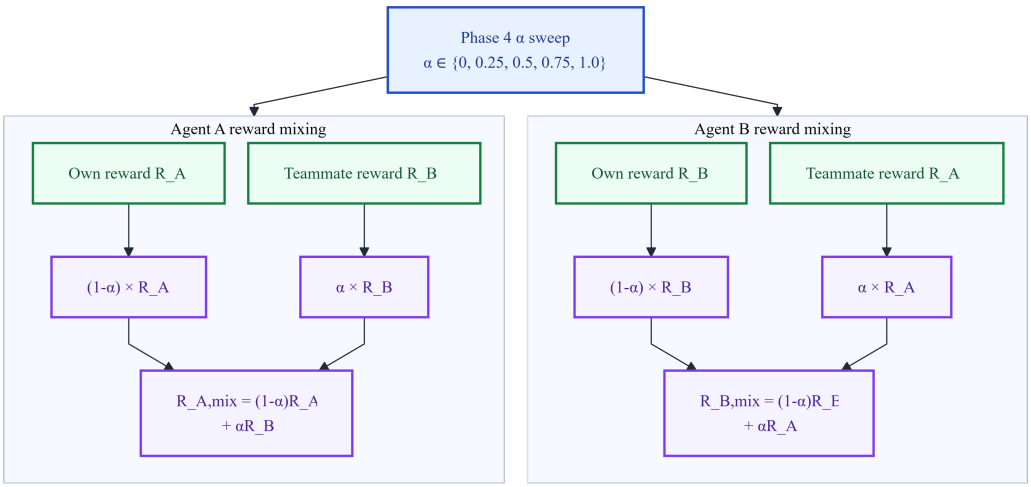

Figure 2.1: α reward mixing for Agents A and B under Phase 4. Each agent's mixed reward is a weighted combination of its own reward and its teammate's reward.

2.3.9 Phase 5: Non-zero-sum game design

Phases 2 to 4 use a two-agent zero-sum race where one agent's gain is always the other's loss. Phase 5 breaks this constraint by adding a fixed Base adversary as a third competitor, so both DQN agents can improve simultaneously by outperforming the adversary. An ablation within Phase 5 replaces each DQN agent with a Base agent in turn, isolating whether game structure or policy quality drives any observed improvement.

2.3.10 Phases 6 and 7: Multi-team scaling

Phase 6 extends the non-zero-sum setting to four DQN agents grouped as two teams of two, plus a Base adversary (five total agents on track). This tests whether the cooperative advantage from Phase 5 survives scaling.

Phase 7 splits into two sub-phases. Phase 7A tests three DQN agents plus the Base adversary (four total) to identify the agent count at which cooperation transitions from beneficial to harmful. Phase 7B revisits the five-agent setting with a low-α sweep to test whether any α value can

recover cooperative performance at that scale. Together, Phases 6 and 7 test whether Phases 4 and 5's findings generalise beyond two agents.

2.3.11 Phase 8: Curriculum Alpha (α) scheduling

Phase 8 applies a three-stage training schedule rather than a fixed α throughout. Agents train for 100 episodes at $\alpha = 0$ to establish individual policies, α increases linearly over 300 episodes to the target, and a final 100 episodes hold at the target. This tests whether the passivity failure at fixed $\alpha = 0.75$ reflects cooperative reward overwriting individual policies before they stabilise, as opposed to a structural incompatibility between cooperative incentives and independent learning. Phase 8 splits into Phase 8A (three DQN + Base) and Phase 8B (four DQN + Base) to test the curriculum at both scaling boundaries identified in Phases 6 and 7. The load-bearing element is the warm-up, ramp, and consolidation structure rather than the specific episode counts (Section 2.2.4).

2.3.12 Phase 9: LLM semantic baseline

Phase 9 introduces a zero-shot large language model as a non-learning comparator. The LLM is integrated into the simulator and queried automatically once per lap. Its prompt is programmatically constructed from simulator state and track configuration, but uses a fixed hand-authored system template that encodes domain knowledge, approximate probability heuristics, and strategic guidance. The model outputs a structured zone plan, which the simulator then executes using live gap information. No training, replay buffer, or gradient update is used.

The purpose is to establish whether domain reasoning with structured priors can match a trained DQN agent within this simulator's scope. Parity would indicate the learning problem is narrow enough to be solved by prompt-encoded reasoning, qualifying how broadly the MARL findings can be read, while a meaningful gap would indicate that learned policies capture strategic structure prompt-encoded reasoning cannot.

2.3.13 Phase 10: Difference rewards

Phase 10 tests difference rewards as an alternative cooperative formulation at the three-DQN-plus-Base configuration (four total agents). α mixing is a heuristic that does not formally separate individual contributions, so its passivity failure is fundamentally a credit assignment problem. Difference rewards address this by giving each agent credit only for its counterfactual contribution to collective performance (Wolpert and Tumer, 2001). Computing the true counterfactual is costly, so a mean-field approximation is used where each agent's reward is its position delta plus the deviation from the team mean.

$$R_{\text{diff},i} = \Delta_i + (\Delta_i - \bar{\Delta}_{\text{team}})$$

Whether this approximation preserves the cooperative gradient property is the central question of Phase 10.

2.4 Project management

2.4.1 Sprint structure and deliverable mapping

Each sprint covers one phase and produces three artefacts. A protocol documents the phase design and gate criteria, an analysis captures results and interpretation, and a dataset artefact contains the evaluation outputs.

The five project deliverables map to sprint groups. Deliverable 3 (minimal viable product with basic metrics) corresponds to Sprints 0 and 1. Deliverable 4 (expanded state space, varied parameters, and algorithm comparison) corresponds to Sprints 2 through 9. Deliverable 5 (final report with comparative analysis) is the present document. The gate structure means each deliverable is verifiable at its own checkpoint rather than assessed only at final submission.

2.4.2 Version control and gate discipline

Version control uses Git with one feature branch per phase. A phase branch starts from the current main, carries all development for that phase, and merges back only after the phase's gate or validation criteria are met. This rule means the main branch always reflects a gate-passed simulator state, and any commit on main can be checked out and reproduced in full.

Commit messages document phase-level findings at meaningful checkpoints. These include zone behaviour observations, collapse events, and implementation details that affected results. The commit history therefore doubles as an engineering log, with each phase's development trail preserved for later audit. Every run additionally embeds its executed configuration, seed assignments, and training and evaluation budgets into its output metrics, so any experiment can be reproduced from that record alone (Appendix G).

2.4.3 Research question independence

The phased structure is not tied to any one research question. The simulator, gate machinery, and evaluation infrastructure are built around stochasticity level, agent count, and reward formulation without picking one as primary. New phases extend the existing ones rather than replace them, and completed phases remain valid whatever direction the experimental matrix takes.

Chapter 3

Implementation and Validation

3.1 System architecture

3.1.1 Module overview

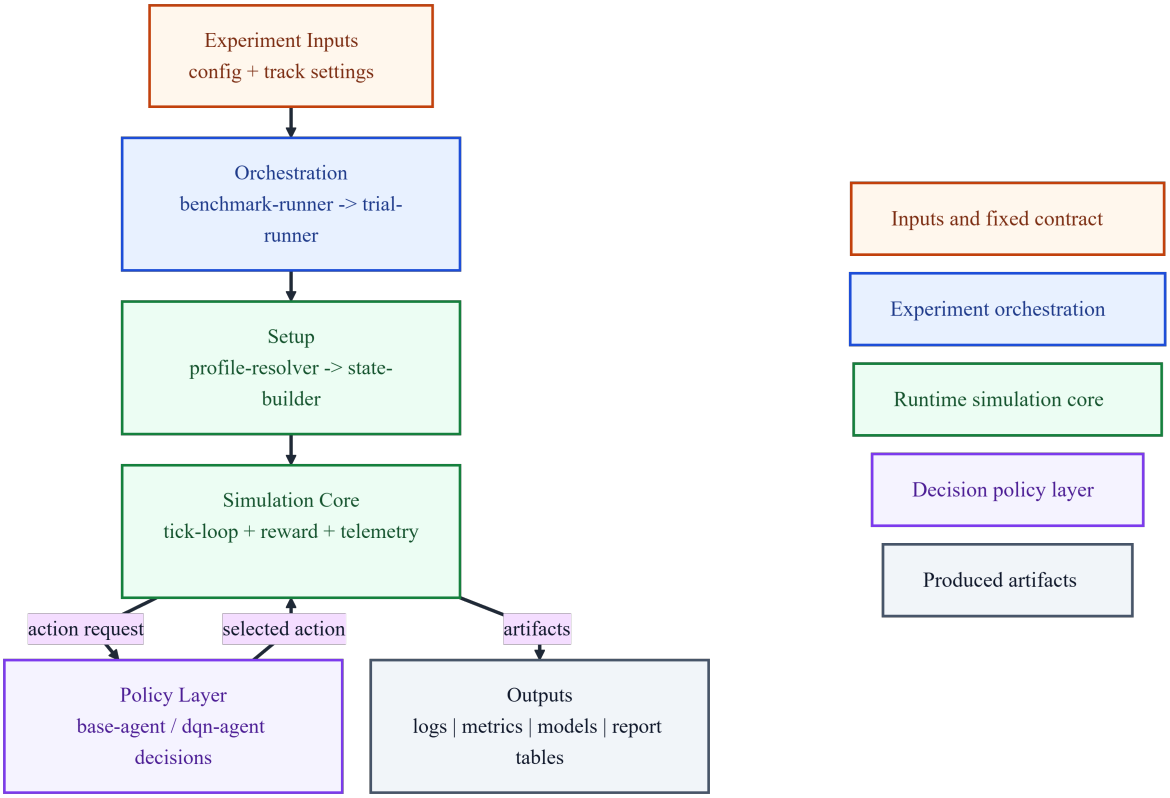

Figure 3.1: System architecture overview. Configuration and track settings flow downward through experiment orchestration and runtime setup to the simulation core. The simulation core exchanges action requests and decisions with the policy layer and writes run artefacts to the output layer.

The simulator is built as a layered Python system. A configuration and orchestration layer resolves the active complexity profile and sequences calls across the algorithm comparison matrix. A simulation core owns the tick-by-tick race loop and queries agents for actions at each overtaking zone. All agents implement a common interface returning a structured action, so the engine is variant-agnostic. Per-module responsibilities and the UML class diagram are in Appendix F.

3.1.2 Config-driven design as a fairness control

All parameters that affect the experimental outcome are externalised to a single configuration file. This covers hyperparameters, the reward component schema and weights, the observation feature list, stochasticity presets, evaluation protocol budgets, and seed sets. No algorithm implementation reads or modifies these values directly. They are injected at construction time.

This design ensures that the only difference between a vanilla and a double DQN run is the learning algorithm itself. An external reviewer can fully reconstruct any comparison from the recorded output of each run, which embeds the executed configuration, budgets, and seed assignments.

3.2 Agent implementations

3.2.1 DQN-family agents

All four DQN variants are parameterisations of a single agent class, selected via a config field. The training loop, action space, epsilon schedule, and feedback interface are shared across variants, and only the variant-specific elements differ. The shared training loop structure is illustrated in Appendix E, and the full variant formulations and hyperparameters are in Appendix D.

Vanilla DQN. The vanilla variant uses a small feedforward neural network to approximate the Q-value function, which predicts the expected long-term reward for each action in a given state. Past decisions are stored in a replay buffer and resampled to break the correlation in training data, and a slowly updated target network provides a stable prediction target during learning. This is the baseline against which all other variants are compared (Mnih et al., 2015).

Double DQN. Vanilla DQN tends to be over-optimistic about how good certain actions are, especially when rewards are noisy. Double DQN splits the decision into two steps. The online network picks the best next action, and the older target network grades it. This decouples action selection from evaluation, so random noise in the estimates stops compounding into systematic overestimation (van Hasselt et al., 2016). The architecture and uniform replay are otherwise the same as vanilla.

Dueling DQN. Dueling DQN makes learning more efficient by recognising that action choice does not always matter. In many states (for example, when the car ahead is far away), every action leads to roughly the same outcome, so the agent only needs to learn that the state is fine. The dueling architecture splits the network into two output heads, a state-value head that estimates how good a state is overall and an action-advantage head that estimates how much better or worse each specific action is relative to the average. This architecture improves learning efficiency in states where the choice of action has little effect on the outcome, which is common at overtaking zones where the car ahead is far away and HOLD is clearly dominant (Wang et al., 2016).

Rainbow-lite. Rainbow-lite combines the three previous variants with two additions, which together make it the strongest in the DQN family. It uses double DQN target computation, the

dueling architecture, prioritised experience replay (PER), and multi-step returns. PER replays transitions that caused large prediction errors more often than transitions the agent already understands well, focusing learning effort where it is most useful (Schaul et al., 2016). Multi-step returns let each update use rewards from the next three steps rather than just one, which propagates useful information through the network faster (Hessel et al., 2018).

Table 3.1 summarises the distinguishing properties of each variant.

Table 3.1: DQN-family variant properties

Variant	**Network**	**Double target**	**PER**	**n-step**
Vanilla	MLP	No	No	1
Double	MLP	Yes	No	1
Dueling	Dueling MLP	No	No	1
Rainbow-lite	Dueling MLP	Yes	Yes	3

3.2.2 Base agent

The Base Agent is a fixed, reproducible heuristic opponent used as the non-learning comparator in Phase 2 and from Phase 5 onwards as the common adversary in non-zero-sum configurations. Holding its policy constant across episodes isolates DQN-side learning from opponent drift, so any change in win rate attributes cleanly to the learning agent. Its decision logic is specified in Appendix G.

3.2.3 Shared training infrastructure

The epsilon-greedy exploration schedule is shared across variants. The agent selects a random action with probability ϵ and the best-known action otherwise, with ϵ decaying from fully exploratory at the start of training to near-converged at the end. The Adam optimiser (Kingma and Ba, 2015) updates network weights across all variants. Adam adapts the step size for each network parameter based on the history of its gradients, which stabilises training when gradients vary in magnitude. The specific decay schedule, learning rate, target network synchronisation frequency, and batch size are given in Appendix D.

The shared hyperparameters were selected via automated single-parameter search against the Phase 2 single-agent contract (Appendix D). Tuning targeted single-agent stability rather than multi-agent dynamics, which keeps DQN-variant comparisons fair within Phase 2 and avoids tuning bias that could favour any particular variant in later phases.

3.3 Benchmark infrastructure

Benchmark runs use a single-trial runner that executes one train-then-evaluate cycle for a given algorithm variant and writes a structured metrics file with the win rate, fairness diagnostics, and per-zone behavioural breakdown. A benchmark matrix orchestrator sequences these trials across the full comparison matrix (algorithm $\times$ seed $\times$ stochasticity level) and checks fairness against

a declared contract, aborting on any violation. Every race additionally produces a JSON Lines entry recording per-decision telemetry, with arithmetic consistency of the reward decomposition verified by the Phase 0 integrity tests. Full specifications for the runner, orchestrator, and telemetry schema are in Appendix H and Appendix C.

3.3.1 MARL evaluation runner

A separate evaluation runner handles MARL trials from Phase 3 onwards. Each trial loads two trained DQN policies into fixed evaluation slots A1 and A2, where Agent 1 and Agent 2 refer to the policies loaded into those slots. The primary metric is the fraction of evaluation races in which Agent 1 finishes ahead of Agent 2. The runner also computes a zone differentiation index (the mean absolute difference in per-zone attempt rates between the two agents) and a risk differentiation index (the mean absolute difference in attempt-conditioned risk proportions). A non-stationarity drift metric records the change in Agent 1's win rate between the first and last third of the evaluation window, testing whether the advantage relationship is stable. From Phase 5 onwards the runner records team-level metrics including the joint beat-base rate, the fraction of races in which all active DQN agents finish ahead of the Base adversary, which measures cooperative performance rather than relative DQN performance.

A crossplay protocol validates the evaluation setup itself. In the self-play condition, both slots are loaded with the same trained weights, so any systematic deviation from a 0.500 A1 win rate indicates positional bias. In the swapped condition, the A1 and A2 model files are exchanged, isolating policy quality from slot assignment. Results are reported in Appendix I and incorporated into the validity threat discussion in Section 4.7.2.

3.3.2 MARL and phase extensions

The codebase was extended phase by phase to support multi-agent learning (Phase 3), α reward blending (Phase 4), team grouping (Phase 6), curriculum scheduling (Phase 8), mean-field difference rewards (Phase 10), and the Claude Haiku LLM agent (Phase 9). All extensions were controlled through configuration with no changes to agent learning code. The full specification is in Appendix K.

3.3.3 Challenges and resolutions

Three significant challenges were encountered during implementation. First, the initial Base Agent implementation had a missing gap attribute that caused near-zero attempt probability. This surfaced during Phase 0 testing rather than being caught by face-value inspection, validating the value of automated integrity tests. Second, the telemetry reward arithmetic check revealed a reward-sum accounting error in the reward decomposition that would have remained hidden without the dedicated arithmetic verification field. Third, a Phase 10 configuration specified the wrong algorithm variant. The error was caught during analysis by reading the algorithm field from the telemetry output rather than the config file, confirming the value of config snapshots

embedded in run metrics. Each case points to the same methodological pattern. Integrity machinery that reads from output rather than input catches silent-failure bugs that face-value checks miss.

3.4 Gate validation summary

Phases 0 and 1 served as pre-benchmark gates. Phase 0's 12-test integrity suite passed across all three groups after two defects were caught and corrected during development. Phase 1's single-agent smoke evaluation cleared the 0.50 win-rate threshold across three s0 seeds and one seed at each of s1 and s2, with the CI lower bound also clearing 0.50. The risk-distribution shift observed under increasing stochasticity is consistent with Q-value overestimation under noisy rewards (van Hasselt et al., 2016), which motivates the Phase 2 double DQN comparison. Full test inventories, defect logs, and per-zone breakdowns are in Appendix C.

Chapter 4

Results, Evaluation and Discussion

This chapter reports the empirical findings of Phases 2 to 10 organised around the four research questions from Chapter 1. Phase 9 is reported separately as a validity check on the investigation.

4.1 Preliminary validation

The three preliminary gates all passed. Phase 0 system integrity passed all 12 tests after 2 defects were corrected (Appendix C). Phase 1 smoke validation cleared the 0.50 gate across all s0 seeds with CI lower bound also clearing 0.50. Phase 2 established the DQN-family single-agent ranking at s0 as rainbow-lite (0.835, CI95 0.813-0.856), double (0.798), vanilla (0.746, CI95 0.723-0.769), and dueling (0.724), with rainbow-lite's interval not overlapping vanilla.

Rainbow-lite's s0 to s2 change is +0.005, consistent with PER and double-target correction reducing value instability (Schaul et al., 2016; van Hasselt et al., 2016). Dueling ranked below vanilla, suggesting value-advantage decomposition did not help in this two-agent setting. Rainbow-lite is therefore the leading single-agent substrate candidate, though the single-agent contract cannot establish MARL viability on its own. The Phase 3 collapse ablation in the next section both answers RQ1 and confirms the substrate choice under concurrent learning.

4.2 RQ1: What strategic behaviours emerge and how robust are they?

Phase 3 tested four DQN-family variants under concurrent learning at $\alpha = 0$, where two instances of the same variant race each other in a zero-sum setting. Each variant probes a different hypothesis about what drives MARL pathologies here. Vanilla acts as the control, double isolates maximum-operator overestimation, dueling isolates representational structure, and rainbow-lite tests whether multiple coupled issues (overestimation, sampling imbalance, credit propagation) need addressing together. Collapse rates across nine trials per algorithm are in Table 4.1.

Table 4.1: Phase 3 collapse rates by algorithm (nine trials per algorithm)

Algorithm	**s0**	**s1**	**s2**	**Total collapse**
Vanilla DQN	1/3	1/3	2/3	4/9 (44%)
Double DQN	1/3	0/3	0/3	1/9 (11%)
Dueling DQN	1/3	1/3	2/3	4/9 (44%)
Rainbow-lite	0/3	0/3	0/3	0/9 (0%)

The ablation picks out two distinct failure modes. Double DQN addresses the first and largest source of error. In this domain, early training produces many failed attempts for the trailing

agent punctuated by occasional lucky successes at hard zones, which inflates Q-values for the chosen action under vanilla's target computation. Double target correction dampens this by decoupling action selection from evaluation, which removes a real first-order source of error that affects many runs. Double eliminates collapse entirely at s1 and s2, with only one collapse at s0.

Dueling DQN fails to help and has the same collapse rate as vanilla. Its architectural advantage targets a problem this simulator does not really have. Dueling's decomposition of $Q(s, a)$ into state-value and action-advantage works best when many actions are near-equivalent in most states, so that separating "is this state good?" from "which action matters?" reduces learning variance. The overtaking decision is the opposite case. HOLD versus ATTEMPT, and risk level within ATTEMPT, are strongly differentiated in outcome. The action split is where most of the information sits. In this setting, dueling's decomposition can actually sharpen bad action preferences rather than smooth them, making the agent converge to a bad strategy faster once it drifts into one.

Rainbow-lite closes the remaining gap from 11 per cent collapse under double to zero. Prioritised experience replay (PER) is the load-bearing addition in this marginal improvement. Dueling is neutral or negative in this domain (Phase 3 ablation), and multi-step returns are inferentially secondary because they are not separately isolated in the ablation design. PER addresses the second failure mode, which is sampling imbalance in the replay buffer. After double has removed the broad overestimation problem, what remains is a residual where the buffer accumulates systematic bias from positional asymmetry. Early failed attempts by the trailing agent produce $\langle$attempt, negative$\rangle$ transitions that dominate uniform replay. Q(hold) then rises above Q(attempt), and shared epsilon decay locks in passivity before exploration can recover. PER breaks this lock-in by up-weighting high TD-error transitions, which are disproportionately the rare successful attempts that need to be learned from. The account is grounded in the ablation logic and in the observed collapse pattern, where one agent gets all zone encounters and makes zero attempts. The replay buffer contents are not directly instrumented during training, so the account is inferred from the external signature rather than verified internally.

The ablation hierarchy therefore reflects two separable contributions. Double target correction removes the large first-order overestimation error that drives most collapses, and PER closes the residual sampling trap that double cannot reach. To test whether collapse under vanilla was simply a case of the training horizon being too short, vanilla was extended to 750 episodes. Collapse consistently worsened across seeds rather than recovering, confirming that once the loop engages it is self-reinforcing within the training horizon (Foerster et al., 2017; Matignon et al., 2012). Rainbow-lite is therefore the only variant that survives concurrent learning and is used as the substrate for all later phases, with double DQN retained in the Phase 3 family ablation and vanilla in Phase 10 as a weak-algorithm comparator.

Where the algorithm prevents collapse, agents develop differentiated zone coverage and opposing risk profiles without any shared coordination. A representative rainbow-lite trial at s2 with 750 episodes reaches a zone differentiation index of 0.405, with Agent 1 focusing on La Source, Les Combes, and Bruxelles with a conservative bias, and Agent 2 covering five zones with an aggressive bias. The pattern echoes the hawk-dove game from evolutionary game theory (May-

nard Smith and Price, 1973), where two players contesting a shared resource settle into asymmetric roles (one aggressive, one passive) as opposed to both playing aggressively at the cost of mutual destruction. Here the equilibrium emerges through iterated best-response learning, where each agent gradually adjusts its policy to be optimal against the current policy of the other, converging to a stable joint profile (Buşoniu et al., 2008). Collapsed trials produce no such structure. One agent stops competing entirely and the other wins unopposed, matching the degenerate equilibrium predicted for non-cooperative two-agent Q-learning under positional disadvantage (Tan, 1993).

Specialisation depends on training budget, though with high across-seed variance. Seed 202 rises from 0.188 at 500 episodes to 0.405 at 750 (the canonical case), seed 101 from 0.022 to 0.163, and seed 303 from 0.064 to 0.054 (no recovery). The two diagnostic seeds (404 and 505) sampled only at 500 episodes bracket a wide range (0.043 and 0.520), showing some initial conditions reach high differentiation early while others do not. The pattern is budget-sensitive rather than budget-determined (Figure 4.1). PER needs sufficient zone-specific transitions to produce reliable relative priorities, but initialisation variance also matters for how quickly this stabilises.

A crossplay protocol at s2 revealed an 11 percentage point structural bias against the A1 evaluation slot, affecting raw A1 win rates but not collapse classification. After applying the correction, four of five rainbow-lite seeds sit at or above parity, and the apparent A2 dominance at s2 largely dissolves (full arithmetic in Appendix I). This supports the evaluation-bias interpretation as opposed to a systematic policy effect. Non-stationarity rather than stochasticity dominates Phase 3. Vanilla and dueling collapse rises from 1/3 at s0 to 2/3 at s2, showing stochasticity accelerates but does not cause collapse (Foerster et al., 2017).

RQ1 answer. H1 is confirmed for collapse-resistant algorithms. In all nine rainbow-lite trials and eight of nine double DQN trials, asymmetric strategic differentiation emerges without explicit coordination. Rainbow-lite produces hawk-dove risk partitioning most clearly, while double DQN's non-collapsed trials sometimes show zone-avoidance asymmetry or wrong-zone fixation instead. Vanilla and dueling collapse in 44 per cent of trials and produce no strategic structure at all, so H1 holds only where the algorithm survives the non-stationarity challenge. Two qualifications apply. Specialisation depends jointly on training budget and initialisation variance rather than budget alone, and non-stationarity from concurrent learning is the dominant challenge as opposed to environmental stochasticity.

4.3 RQ2: How does incentive structure alter strategy?

Phase 4 ran the α sweep in the two-agent zero-sum game. Collapse is low at $\alpha \leq 0.5$, sharp between $\alpha = 0.5$ and $\alpha = 0.75$, and complete at $\alpha = 1.0$ (Table 4.2, Figure 4.2). At $\alpha \geq 0.75$ the cooperative reward term dominates the individual gradient, turning successful overtakes into net penalties and driving both agents to passivity (Claus and Boutilier, 1998). The exact 8/9 figure at $\alpha = 0.75$ is numerically imprecise under the three-seed design, though the qualitative pattern is robust.

Table 4.2: Phase 4 collapse rates by α in two-agent zero-sum game (9 trials per α)

α	**Collapse rate**	**Competitive trials**
0.00	0/9 (0%)	100%
0.25	1/9 (11%)	89%
0.50	1/9 (11%)	89%
0.75	8/9 (89%)	11%
1.00	9/9 (100%)	0%

Two secondary patterns within the threshold qualify how α acts rather than just where it acts. Zone convention formation is non-monotonic, peaking at $\alpha = 0.25$ and declining thereafter, and stochasticity's role inverts across the threshold from stabilising at low α to destabilising at high α (Appendix J). Together these show α changing the type of equilibrium rather than the strength of a single one. Low α produces competitive convention with differentiated zone coverage. High α produces an unstable passive equilibrium where both agents learn to hold.

Phase 5 ran the same sweep in the minimal non-zero-sum configuration (two DQN agents plus one Base adversary). The curve is fundamentally different (Table 4.3). Collapse disappears at all α values. Joint beat-base rate follows an inverted-U peaking at $\alpha = 0.75$ with an 83 per cent improvement over the $\alpha = 0$ baseline. There is no destabilisation threshold here, only a performance optimum. Full cooperation at $\alpha = 1.0$ stays active but becomes higher-variance and less consistent than $\alpha = 0.75$, confirming that 0.75 is a genuine optimum as opposed to a saturating plateau.

Table 4.3: Phase 5 joint beat-base rate by α in three-agent non-zero-sum game (9 trials per α)

α	**Joint beat-base**	**Change from $\alpha = 0$**	**Collapse rate**
0.00	0.310	baseline	0%
0.25	0.294	−5%	0%
0.50	0.433	+40%	0%
0.75	0.567	+83%	0%
1.00	0.436	+41%	0%

Cooperation has a minimum effective dose. At $\alpha = 0.25$ joint beat-base drops 5 per cent below the $\alpha = 0$ baseline, so weak sharing is worse than none. Meaningful benefit starts at $\alpha = 0.5$ and peaks at $\alpha = 0.75$. Small cooperative signals plausibly introduce noise into the individual gradient without inducing role partitioning, leaving agents with weakened incentive signal and no coordination benefit. Above $\alpha = 0.5$ the signal is strong enough to drive complementary role development.

The mechanism behind the cooperative gain is redistributive rather than uniform. Agent 2 rises from 63 per cent beat-base at $\alpha = 0$ to 78 per cent at $\alpha = 0.75$, while Agent 1's rate changes less. The joint metric moves because the floor rises. This is consistent with cooperative incentive functioning as a mechanism that closes within-team performance gaps rather than lifting both agents in parallel (Hughes et al., 2018). Strategy and performance metrics also separate. Zone differentiation peaks at low α, risk differentiation increases with α, and joint performance peaks at $\alpha = 0.75$. More differentiation does not produce better performance on its own.

RQ2 answer. H2 predicted a sharp threshold above which shared reward destabilises learning,

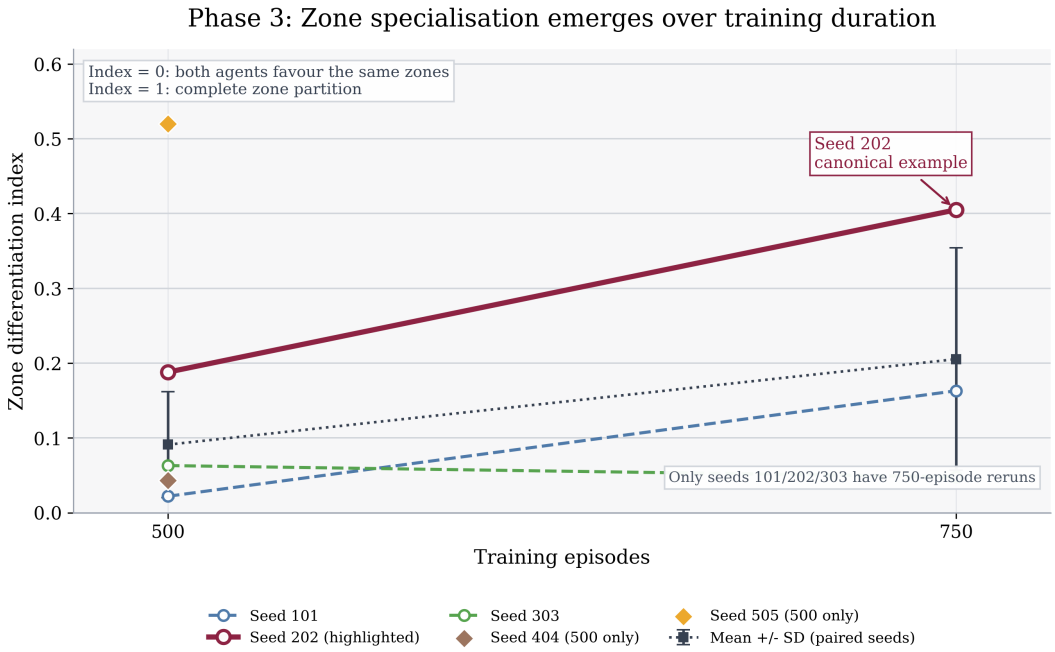

Figure 4.1: Zone differentiation growth for rainbow-lite at s2 showing dependence on training budget.

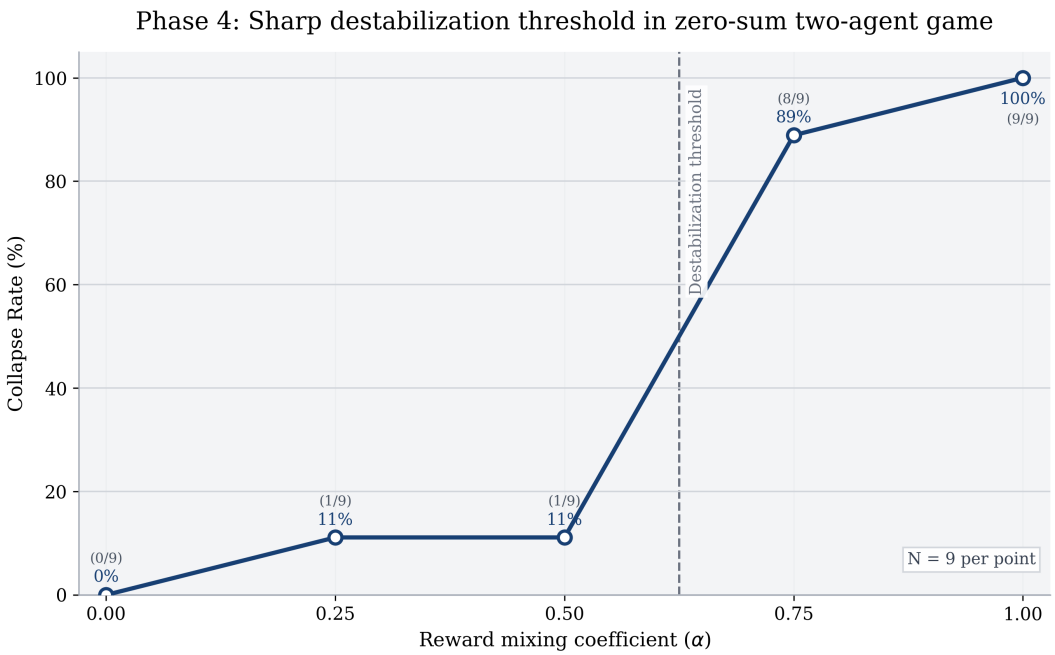

Figure 4.2: Collapse rates by α in the two-agent zero-sum game.

with non-zero-sum structure reversing the effect. The zero-sum part is confirmed at the $\alpha = 0.5$ to $\alpha = 0.75$ boundary, with a non-monotonic convention-formation pattern below and a stochasticity inversion across it. The non-zero-sum part does not simply reverse. The destabilisation disappears, replaced by an inverted-U performance curve with a minimum effective dose between $\alpha = 0.25$ and $\alpha = 0.5$ and an optimum at $\alpha = 0.75$, with gains driven by lifting the weaker agent. H2 is therefore confirmed in its zero-sum part and restated in its non-zero-sum part, because α changes the type of equilibrium as opposed to only the direction of its effect. Whether cooperative advantage survives at larger agent counts is addressed in RQ3.

4.4 RQ3: Does shared incentive produce genuine cooperative advantage at scale?

RQ2 established that cooperative advantage exists in the minimal non-zero-sum setting and produces a sharp performance optimum at $\alpha = 0.75$. RQ3 tests whether this survives scaling, and the failure mode that emerges is importantly different from anything in RQ2. Across Phases 6, 7, and 8, collapse remains at zero in all 72 trials. The system stays active at scale. What disappears is the cooperative surplus. Agents still race and still respond to α, but shared reward stops producing within-team co-improvement.

Phase 6 scaled to four DQN agents in two teams of two plus the Base adversary. Every tested form of cooperation degrades performance. At symmetric $\alpha = 0.75$, joint beat-base drops 31 per cent below the competitive baseline. In the asymmetric condition the cooperative team underperforms against the baseline and also loses directly to the competitive team in head-to-head comparison. Team A at $\alpha = 0.75$ wins only 47.5 per cent of races against Team B at $\alpha = 0.0$, with worse average position and lower both-beat-base (Table 4.4). The Base adversary benefits directly from cooperative-team degradation. Under symmetric $\alpha = 0.75$, Base moves from near last in the competitive baseline to near third in the field ranking. Curriculum scheduling largely restores Base to its original position, confirming that cooperative-team caution under fixed α opens space the Base heuristic can exploit.

Table 4.4: Phase 6 both-beat-base rate by team α condition (9 trials per condition)

Condition	**Both-beat-base**	**Change from 0 vs 0**
Team A $\alpha = 0.0$ vs Team B $\alpha = 0.0$	0.350	baseline
Team A $\alpha = 0.5$ vs Team B $\alpha = 0.5$	0.306	−12%
Team A $\alpha = 0.75$ vs Team B $\alpha = 0.75$	0.243	−31%
Team A $\alpha = 0.75$ vs Team B $\alpha = 0.0$	0.291	−17%

The strongest mechanistic evidence is the persistently negative intra-team position-delta correlation. It is negative in all 36 Phase 6 trials. Shared reward between teammates never becomes genuine within-team co-improvement at this scale, regardless of α or condition. The root cause is causal dilution of the pairwise cooperative signal. Reward sharing remains within teammate pairs, but at four agents each teammate's outcome is now influenced by three competitors rather than one. The pairwise signal carries weaker causal information about the teammate's actions, so updates become noisier and agents learn caution rather than coordination (Agogino and Tumer,

2004).

The observation contract included teammate-aware features (teammate-ahead and teammate-behind indicators) that agents could have conditioned on, yet no team-conditional behaviour emerged. Agents have the information needed for coordination but lack the reward signal to learn from it, so the bottleneck is credit assignment rather than observation-space design.

Phase 7 characterised the scaling boundary. The $\alpha = 0.75$ effect moves from +83% at three agents, to +5% at four agents (Phase 7A, within noise), to −31% at five agents (Phase 6), shown in Figure 4.3. Part of the drop from 0.310 to 0.159 reflects a baseline compounding effect where a fourth agent requires one more DQN to beat Base, pushing the joint rate from p^2 to p^3 territory. The meaningful result is that $\alpha = 0.75$ fails to create any synergy above this compounded baseline. Individual beat-base rates also barely move under α in Phase 7A, staying roughly in a 52 to 61 per cent band while joint beat-base sits at the independence baseline. α changes structure without producing synergy.

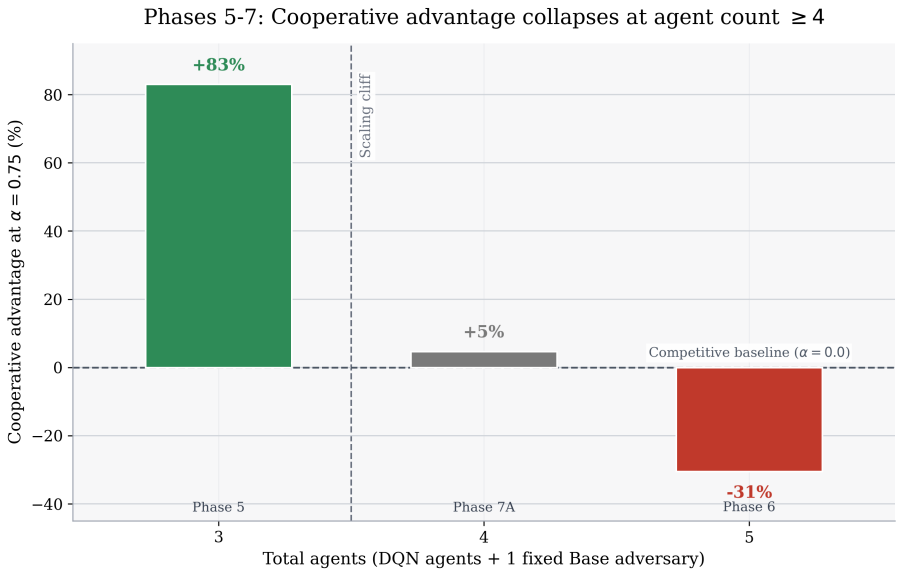

Figure 4.3: Cooperative advantage at $\alpha = 0.75$ across agent counts.

Phase 7B tested whether any α value recovers cooperative performance at five agents. Combined with Phase 6 reference points at $\alpha = 0.0$, 0.5, and 0.75, the full five-agent team-α sweep is essentially monotonic downward (0.350, 0.344, 0.349, 0.315, 0.306, 0.243 at $\alpha = 0.00, 0.10, 0.15, 0.25, 0.50, 0.75$). No α value recovers performance, and the failure is structural rather than parametric. Zone specialisation is necessary but not sufficient for cooperative advantage. At four agents, $\alpha = 0.75$ raised zone differentiation by 63 per cent relative to $\alpha = 0$, yet produced no joint benefit. Attempt rates simultaneously dropped 38 to 64 per cent across zones, so agents responded to the cooperative signal by becoming more cautious rather than coordinating effectively.

Phase 8 tested whether curriculum scheduling recovers cooperative performance. At five agents (Phase 8B) curriculum recovered 69 per cent of the performance loss (joint beat-base rising from 0.243 to 0.318 against the 0.350 baseline), and excessive caution fell 86 to 89 per cent. At four

agents (Phase 8A) curriculum almost eliminated the caution pathology. Curriculum therefore solves the training-order damage that high fixed α causes, yet in no condition does it exceed the competitive baseline. The intra-team correlation becomes even more negative under curriculum (-0.189 in Phase 8B), meaning agents improve individually while actively anti-coordinating. Curriculum cleanly separates two failure modes often discussed together (Panait and Luke, 2005; Zhao et al., 2025), resolving training-order collapse while leaving credit-assignment failure untouched. At four or more agents, reward-level interventions appear insufficient, pointing toward architectural changes such as CTDE-style methods as the next plausible route (Rashid et al., 2018).

RQ3 answer. H3 is confirmed. Phase 7B's monotonic downward curve, Phase 8's recovered-but-anti-coordinating outcome, and the persistently negative intra-team correlation across all scaling phases together provide direct evidence. The scaling failure manifests as active anti-coordination as opposed to passive collapse. Agents remain active, α still changes zone allocation, and curriculum restores aggressive play, but none of these changes produce positive within-team coupling. The curriculum clause of H2 finds support in Phase 8's 69 per cent recovery, which is real but insufficient for cooperation. The structural boundary sits at four total agents in this simulator.

4.5 RQ4: Do alternative cooperative reward formulations preserve the advantage?

Phase 10 tested the mean-field difference reward approximation (Section 2.3.13) at the four-total-agent configuration (3 DQN + Base), matching the Phase 7A setup where α-mixed cooperation had already failed. The test is therefore whether an alternative cooperative formulation recovers advantage at the scaling boundary where α mixing broke down.

The implemented formula has two structural properties that together explain why it cannot produce cooperation. First, the total reward mass is conserved because the sum of teammate rewards equals the sum of their individual deltas, so the formula redistributes credit rather than adding cooperative mass. Second, differentiating any agent's reward with respect to a teammate's delta gives $-1/N$, which is negative. Differentiating with respect to that same agent's own delta gives $(2N-1)/N$, which is amplified relative to the identity. The approximation therefore amplifies own-performance incentive and penalises teammate improvement.

Table 4.5: Phase 10 joint beat-base rate under difference rewards versus IQL baseline at 4 total agents

Condition	**Joint beat-base**	**Change from IQL**
IQL baseline (rainbow-lite, $\alpha = 0$)	0.159	baseline
Difference rewards (vanilla DQN)	0.087	-45%
Difference rewards (rainbow-lite)	0.133	-16%

The empirical result matches the mechanism. Difference rewards rank last among the cooperative formulations tested at the four-agent boundary. The original Phase 10 run used vanilla DQN

by mistake, producing 0.087 joint beat-base. The corrected rainbow-lite rerun rose to 0.133 but still sits below the 0.159 IQL baseline, confirming the negative result survives removal of the algorithm confound. The 31 per cent relative algorithm effect (vanilla to rainbow-lite under difference rewards) exceeds the 16 per cent relative formula effect (IQL to rainbow-lite difference rewards), so algorithm choice remains determining even under a changed reward formula.

The deficit is primarily a within-team coordination failure rather than a failure against Base. Base average position in the corrected rainbow-lite run sits at 2.72, comparable to Phase 7A IQL's 2.69. The joint shortfall reflects the DQN group failing to convert individual competitiveness into joint beat-base success, which is consistent with the structural property analysis. Reward amplification for own-performance keeps individual agents active, but the negative teammate gradient prevents the joint metric from rising.

RQ4 answer. H4 is confirmed. The implemented mean-field difference reward underperforms all tested four-agent baselines because it redistributes credit without adding cooperative mass, amplifies own-performance incentive, and penalises teammate improvement. The result is about this specific approximation, not Wolpert-Tumer difference rewards in general. A formulation preserving the counterfactual property of the original literature may or may not produce cooperation at this scale, which is a question for future work. Within RQ4's scope, cooperative advantage requires a reward formula with positive cooperative gradients.

4.6 Validity check: LLM semantic baseline

Phase 9 (Section 2.3.12) compared trained rainbow-lite against a zero-shot LLM with hand-authored strategic priors, across five seeds with 150 races per seed. Rainbow-lite achieved 0.835, 0.839, and 0.840 at s0, s1, s2 respectively, against the LLM's 0.819, 0.827, and 0.837. The gap is not statistically meaningful. Strategies differed qualitatively. The LLM selected NORMAL at La Source, reserved CONSERVATIVE for difficulty-0.7 zones, avoided Raidillon entirely, and used Stavelot in a pattern rainbow-lite never developed. The parity suggests the simulator's narrow strategy space admits a short near-optimal policy captured by hand-authored priors (Nagpal et al., 2025). This qualifies the scope of the MARL findings rather than invalidating them, and identifies specific RL-specific pathologies (Raidillon overuse, suboptimal Stavelot behaviour) that domain reasoning avoids. Whether the same dynamics hold in richer environments is a question for future work.

4.7 Discussion

4.7.1 Synthesis

The four RQs together characterise cooperative learning in this simulator. Algorithm choice determines whether competitive equilibrium is maintained (RQ1). Reward sharing produces a sharp threshold whose direction reverses with game structure (RQ2). Cooperative advantage

scales only to three agents and cannot be recovered at four or more by reward-level interventions (RQ3). The mean-field difference rewards approximation fails at the four-agent boundary because it redistributes credit without adding cooperative mass (RQ4). Where cooperation emerges, three structural conditions must hold together, namely non-zero-sum game structure, a collapse-resistant algorithm, and a reward formula with positive cooperative gradients. Where any one fails, the system degrades in the characteristic mode associated with that failure.

4.7.2 Validity threats and limitations

Simulator scope. La Source has base success probability 0.8, so early-phase win rates partly reflect exploitation of one dominant zone. The five-lap Spa format produces sparse decisions and may not generalise to circuits with more balanced zone difficulty.

Evaluation limitations. Crossplay reveals an 11 percentage point structural bias against the A1 slot affecting raw Phase 3 win rates (though not collapse classification). Three seeds per cell widens confidence intervals, though it does not affect the non-overlapping Phase 2 s0 ranking.

Hyperparameter tuning scope. Hyperparameters were tuned against the Phase 2 single-agent contract, so the configuration reflects single-agent stability rather than multi-agent dynamics. Different configurations may produce different Phase 3 onwards results.

LLM prompt confound. The Phase 9 LLM used a hand-authored prompt with strategic priors, so parity measures the language model and prompt jointly. A raw-state prompt would be a stronger comparison.

Base Agent as comparator. Beat-base rates are relative to a specific heuristic and would shift under a stronger or weaker comparator, though they remain internally consistent across phases.

4.7.3 Future directions

Technical improvements. QMIX addresses the credit assignment barrier at four or more agents with a mixing network preserving cooperative gradients (Rashid et al., 2018; Amato, 2024). A potential-based formulation of difference rewards (Devlin et al., 2014) would test whether the Phase 10 failure is specific to the mean-field approximation or to the concept more broadly.

Direct extensions. Medium-complexity simulators with tyre degradation and pit-stops would test whether LLM parity holds in richer state spaces. Rerunning Phase 7 and 8 with a stronger comparator would test whether the scaling cliff's location is an artefact of comparator strength.

Wider impact. The three structural conditions identified in the synthesis apply to any mixed-motive setting with self-interested agents sharing a common goal. The phased validation methodology is portable across MARL investigations for separating methodological failures from scientific ones.

References

Agarwal, R., Schwarzer, M., Castro, P. S., Courville, A. and Bellemare, M. G. (2021), 'Deep reinforcement learning at the edge of the statistical precipice', *Advances in Neural Information Processing Systems* **34**, 29304–29320.
URL: *https://doi.org/10.48550/arXiv.2108.13264*

Agogino, A. K. and Tumer, K. (2004), Unifying temporal and structural credit assignment problems, *in* 'Proceedings of the 3rd International Joint Conference on Autonomous Agents and Multiagent Systems (AAMAS-04)', IEEE, pp. 980–987.
URL: *https://dl.acm.org/doi/10.5555/1018410.1018852*

Amato, C. (2024), 'An introduction to centralised training for decentralised execution in cooperative multi-agent reinforcement learning'. arXiv preprint arXiv:2409.03052.
URL: *https://doi.org/10.48550/arXiv.2409.03052*

Berner, C., Brockman, G., Chan, B., Cheung, V., Dębiak, P., Dennison, C., Farhi, D., Fischer, Q., Hashme, S., Hesse, C., Józefowicz, R., Gray, S., Olsson, C., Pachocki, J., Petrov, M., de Oliveira Pinto, H. P., Raiman, J., Salimans, T., Schlatter, J., Schneider, J., Sidor, S., Sutskever, I., Tang, J., Wolski, F. and Zhang, S. (2019), 'Dota 2 with large scale deep reinforcement learning', *arXiv preprint arXiv:1912.06680* .

Buşoniu, L., Babuška, R. and De Schutter, B. (2008), 'A comprehensive survey of multiagent reinforcement learning', *IEEE Transactions on Systems, Man, and Cybernetics, Part C: Applications and Reviews* **38**(2), 156–172.
URL: *https://doi.org/10.1109/TSMCC.2007.913919*

Claus, C. and Boutilier, C. (1998), The dynamics of reinforcement learning in cooperative multiagent systems, *in* 'Proceedings of the 15th National Conference on Artificial Intelligence (AAAI-98)', AAAI Press, pp. 746–752.
URL: *https://www.cs.toronto.edu/ cebly/Papers/download/multirl.pdf*

Devlin, S., Yliniemi, L., Kudenko, D. and Tumer, K. (2014), Potential-based difference rewards for multiagent reinforcement learning, *in* 'Proceedings of the 13th International Conference on Autonomous Agents and Multiagent Systems (AAMAS-14)', IFAAMAS, pp. 165–172.
URL: *https://core.ac.uk/outputs/103633822/*

Du, Y., Leibo, J. Z., Islam, U., Willis, R. and Sunehag, P. (2023), 'A review of cooperation in multi-agent learning', *arXiv preprint arXiv:2312.05162* .
URL: *https://doi.org/10.48550/arXiv.2312.05162*

Fieni, G., Wüthrich, J., Neumann, M.-P. and Onder, C. H. (2026), 'Learning-based multi-agent race strategies in formula 1', *arXiv preprint arXiv:2602.23056* .
URL: *https://doi.org/10.48550/arXiv.2602.23056*

Foerster, J., Nardelli, N., Farquhar, G., Afouras, T., Torr, P. H. S., Kohli, P. and Whiteson, S. (2017), Stabilising experience replay for deep multi-agent reinforcement learning, *in* 'Proceedings of the 34th International Conference on Machine Learning', Vol. 70 of *Proceedings of Machine Learning Research*, PMLR, pp. 1146–1155.
URL: *https://proceedings.mlr.press/v70/foerster17b.html*

Heilmeier, A., Graf, M. and Lienkamp, M. (2018), A race simulation for strategy decisions in circuit motorsports, *in* '2018 21st International Conference on Intelligent Transportation Systems (ITSC)', pp. 2986–2993. IEEE ITSC 2018.
URL: *https://doi.org/10.1109/ITSC.2018.8570012*

Heilmeier, A., Thomaser, A., Graf, M. and Betz, J. (2020), 'Virtual strategy engineer: Using artificial neural networks for making race strategy decisions in circuit motorsport', *Applied Sciences* **10**(21), 7805.
URL: *https://doi.org/10.3390/app10217805*

Henderson, P., Islam, R., Bachman, P., Pineau, J., Precup, D. and Meger, D. (2018), 'Deep reinforcement learning that matters', *Proceedings of the AAAI Conference on Artificial Intelligence* **32**(1).
URL: *https://doi.org/10.48550/arXiv.1709.06560*

Hernandez-Leal, P., Kartal, B. and Taylor, M. E. (2019), 'A survey and critique of multiagent deep reinforcement learning', *Autonomous Agents and Multi-Agent Systems* **33**(6), 750–797.
URL: *https://doi.org/10.1007/s10458-019-09421-1*

Hessel, M., Modayil, J., van Hasselt, H., Schaul, T., Ostrovski, G., Dabney, W., Horgan, D., Piot, B., Azar, M. and Silver, D. (2018), Rainbow: Combining improvements in deep reinforcement learning, *in* 'Proceedings of the AAAI Conference on Artificial Intelligence', Vol. 32.
URL: *https://doi.org/10.1609/aaai.v32i1.11796*

Hughes, E., Leibo, J. Z., Phillips, M., Tuyls, K., Dueñez-Guzman, E. A., García Castellàneda, A., Dunning, I., Zhu, T., McKee, K. R., Koster, R., Roff, H. and Graepel, T. (2018), Inequity aversion improves cooperation in intertemporal social dilemmas, *in* 'Advances in Neural Information Processing Systems 31', pp. 3330–3340.
URL: *https://doi.org/10.48550/arXiv.1803.08884*

Karpathy, A. (2024), 'Autoresearch: AI-driven automated machine learning improvement'. Open-source tool, GitHub repository.
URL: *https://github.com/karpathy/autoresearch*

Kingma, D. P. and Ba, J. (2015), Adam: A method for stochastic optimization, *in* 'International Conference on Learning Representations (ICLR)'.

Liu, X., Fotouhi, A. and Auger, D. (2024), 'Formula-e multi-car race strategy development—a novel approach using reinforcement learning', *IEEE Transactions on Intelligent Transportation Systems* **25**(8), 9524–9534.
URL: *https://doi.org/10.1109/TITS.2024.3389155*

Matignon, L., Laurent, G. J. and Le Fort-Piat, N. (2012), 'Independent reinforcement learners in cooperative Markov games: A survey regarding coordination problems', *The Knowledge Engineering Review* **27**(1), 1–31.
URL: *https://doi.org/10.1017/S0269888912000057*

Maynard Smith, J. and Price, G. R. (1973), 'The logic of animal conflict', *Nature* **246**(5427), 15–18.

Mnih, V., Kavukcuoglu, K., Silver, D., Rusu, A. A., Veness, J., Bellemare, M. G., Graves, A., Riedmiller, M., Fidjeland, A. K., Ostrovski, G., Petersen, S., Beattie, C., Sadik, A., Antonoglou, I., King, H., Kumaran, D., Wierstra, D., Legg, S. and Hassabis, D. (2015), 'Human-level control through deep reinforcement learning', *Nature* **518**(7540), 529–533.
URL: *https://doi.org/10.1038/nature14236*

Nagpal, K. et al. (2025), Leveraging large language models for effective and explainable multi-agent credit assignment, *in* 'Proceedings of the 24th International Conference on Autonomous Agents and Multiagent Systems', IFAAMAS.
URL: *https://doi.org/10.48550/arXiv.2502.16863*

Ning, Z. and Xie, L. (2024), 'A survey on multi-agent reinforcement learning and its application', *Journal of Automation and Intelligence* **3**(2), 73–91.

Panait, L. and Luke, S. (2005), 'Cooperative multi-agent learning: The state of the art', *Autonomous Agents and Multi-Agent Systems* **11**(3), 387–434.
URL: *https://doi.org/10.1007/s10458-005-2631-2*

Piccinotti, D., Likmeta, A., Brunello, N. and Restelli, M. (2021), Online planning for f1 race strategy identification, *in* 'Workshop on Planning and Reinforcement Learning (PRL) at AAAI'.
URL: *https://prl-theworkshop.github.io/prl2021/papers/PRL2021$_p$aper$_1$.pdf*

Rashid, T., Samvelyan, M., Schroeder de Witt, C., Farquhar, G., Foerster, J. and Whiteson, S. (2018), QMIX: Monotonic value function factorisation for deep multi-agent reinforcement learning, *in* 'Proceedings of the 35th International Conference on Machine Learning', Vol. 80 of *Proceedings of Machine Learning Research*, PMLR, pp. 4295–4304.
URL: *https://proceedings.mlr.press/v80/rashid18a.html*

Sankary, N. and Ostfeld, A. (2018), 'Stochastic scenario evaluation in evolutionary algorithms used for robust scenario-based optimization', *Water Resources Research* **54**(4), 2813–2833.
URL: *https://doi.org/10.1002/2017WR022068*

Schaul, T., Quan, J., Antonoglou, I. and Silver, D. (2016), Prioritized experience replay, *in* 'International Conference on Learning Representations (ICLR)'.
URL: *https://doi.org/10.48550/arXiv.1511.05952*

Silver, D., Hubert, T., Schrittwieser, J., Antonoglou, I., Lai, M., Guez, A., Lanctot, M., Sifre, L., Kumaran, D., Graepel, T., Lillicrap, T., Simonyan, K. and Hassabis, D. (2018), 'A general reinforcement learning algorithm that masters chess, shogi, and Go through self-play', *Science* **362**(6419), 1140–1144.

Tan, M. (1993), Multi-agent reinforcement learning: Independent vs. cooperative agents, *in* 'Proceedings of the 10th International Conference on Machine Learning', Morgan Kaufmann, pp. 330–337.
URL: *https://web.media.mit.edu/ cynthiab/Readings/tan-MAS-reinfLearn.pdf*

van Hasselt, H., Guez, A. and Silver, D. (2016), 'Deep reinforcement learning with double q-learning', *Proceedings of the AAAI Conference on Artificial Intelligence* **30**(1).
URL: *https://doi.org/10.1609/aaai.v30i1.10295*

Vinyals, O., Babuschkin, I., Czarnecki, W. M., Mathieu, M., Dudzik, A., Chung, J., Choi, D. H., Powell, R., Ewalds, T., Georgiev, P., Oh, J., Horgan, D., Kroiss, M., Danihelka, I., Huang, A., Sifre, L., Cai, T., Agapiou, J. P., Jaderberg, M., Vezhnevets, A. S., Leblond, R., Pohlen, T., Dalibard, V., Budden, D., Sulsky, Y., Molloy, J., Paine, T. L., Gulcehre, C., Wang, Z., Pfaff, T., Wu, Y., Ring, R., Yogatama, D., Wünsch, D., McKinney, K., Smith, O., Schaul, T., Lillicrap, T., Kavukcuoglu, K., Hassabis, D., Apps, C. and Silver, D. (2019), 'Grandmaster level in StarCraft II using multi-agent reinforcement learning', *Nature* **575**(7782), 350–354.

Wang, J., Zhang, Y., Hua, Y., Wang, Z., Zhao, Y., Gu, S., He, K., Li, J. and Feng, Y. (2022), Individual reward assisted multi-agent reinforcement learning, *in* 'Proceedings of the 39th International Conference on Machine Learning', Vol. 162 of *Proceedings of Machine Learning Research*, PMLR, pp. 23417–23432.
URL: *https://proceedings.mlr.press/v162/wang22ao.html*

Wang, Z., Schaul, T., Hessel, M., van Hasselt, H., Lanctot, M. and de Freitas, N. (2016), 'Dueling network architectures for deep reinforcement learning', *Proceedings of the 33rd International Conference on Machine Learning* **48**, 1995–2003.
URL: *https://doi.org/10.48550/arXiv.1511.06581*

Wolpert, D. H. and Tumer, K. (2001), 'Optimal payoff functions for members of collectives', *Advances in Complex Systems* **4**(2/3), 265–279.
URL: *https://doi.org/10.1142/S0219525901000188*

Yang, Y., Luo, R., Li, M., Zhou, M., Zhang, W. and Wang, J. (2018), Mean field multi-agent reinforcement learning, *in* 'Proceedings of the 35th International Conference on Machine Learning', Vol. 80 of *Proceedings of Machine Learning Research*, PMLR, pp. 5571–5580.
URL: *https://arxiv.org/abs/1802.05438*

Zhao, Y. et al. (2025), Multi-level advantage credit assignment for cooperative multi-agent reinforcement learning, *in* 'Proceedings of The 28th International Conference on Artificial Intelligence and Statistics', Proceedings of Machine Learning Research, PMLR.
URL: *https://doi.org/10.48550/arXiv.2508.06836*

Appendix A

Self-Appraisal

A.1 Critical self-evaluation

The phased gate structure was the single most effective process decision in the project. Each phase's result determined the next phase's design rather than following a pre-specified plan. Phase 2's finding that rainbow-lite was the only collapse-resistant algorithm in single-agent evaluation meant that Phase 3 could attribute multi-agent collapse to MARL non-stationarity rather than to algorithm fragility. Without this gate, the Phase 3 collapse results would have been ambiguous. The same principle applied at each subsequent gate. Phase 4's zero-sum failure motivated Phase 5's game structure change. Phase 5's success motivated Phase 6's scaling test.

The config-driven fairness control was also effective. Because all experimental parameters were externalised to `config.json` from the start, every comparison was provably identical in all conditions other than the variable under test. An external reviewer can reconstruct any trial from the config snapshot embedded in the metric output file. This discipline was not costly to implement but prevented a class of confounds that would have been difficult to detect and correct retrospectively.

The decision to continue beyond Phase 3 rather than stopping at the first successful result produced the most original contributions. The 83 per cent cooperative improvement in Phase 5 would not have been found if the project had ended at the Phase 3 competitive equilibrium results. The scaling failure in Phase 6, the boundary characterisation in Phase 7, the curriculum decomposition in Phase 8, and the difference rewards analysis in Phase 10 all followed from the decision to keep asking the next question.

The algorithm confound in Phase 10 was the main area that did not work well. The `config.json` file still contained `"algo": "vanilla"` when rainbow-lite was intended for Phase 10A. The first batch of trials therefore tested the wrong algorithm. The error was caught by reading the `"algorithm"` field embedded in the output metric JSON rather than by reviewing the config file before running. The lesson is that config inspection at the start of every experiment batch is a necessary check, not an optional one. Embedding config snapshots in telemetry output is an effective safeguard, but it should not be the only one.

Writing pace was a second area for improvement. Chapters 1 to 3 were drafted while Phases 1 to 3 were active, which made them detailed and timely. Phases 4 to 10 were completed before the corresponding chapters were written, which made the retrospective write-up harder. Writing contemporaneously with experimentation is a better practice.

A.2 Personal reflection and lessons learned

Motorsport has always been an interest of mine, but I came into this project with a limited formal background in race strategy and simulation. The subject sits at an intersection of AI, game theory, and domain knowledge I had not studied systematically. That gap made the project harder, and ultimately more worthwhile.

I proposed my own project rather than selecting a pre-defined one. That gave me creative freedom, but it also removed the guardrails that a well-scoped brief would have provided. At several points during Phase development, the natural next question pulled me further from the original aim. Staying grounded within the scope I had committed to required a discipline that does not always come naturally when curiosity takes over.

The project I submitted is not the project I originally proposed. My initial aim was to study how MARL agents respond to different levels of stochasticity and environmental complexity. That turned out to be a precursor question rather than the central one. Once the stochasticity baseline was established in Phases 0 to 2, the more interesting question emerged. What happens when the incentive structure between agents shifts? The cooperative thinking metric became the actual investigation. Recognising that mid-project and pivoting without losing coherence was the right call, even if it added pressure.

One thread that runs through the project is game theory, a field I had an existing interest in before connecting it explicitly to MARL. The connection became clearer as the experimentation progressed. The alpha sweep in Phase 4 is essentially a parameterisation of the cooperate/defect trade-off from a prisoner's dilemma. Phase 5's non-zero-sum structure breaks the zero-sum constraint in exactly the way game theory predicts cooperation becomes possible. The hawk-dove risk equilibria observed in Phase 3 are a direct empirical instance of a mixed-strategy Nash equilibrium. The Phase 10 difference rewards analysis is a formal question about incentive compatibility, a concept from mechanism design that predates reinforcement learning entirely. Seeing these connections emerge through experimental results rather than being designed in from the start was one of the more satisfying parts of the project.

The most significant conceptual shift was the recognition that negative results are contributions. Phase 10's failure to produce cooperative behaviour via difference rewards was frustrating at first. The mathematical analysis of the gradient inversion transformed it into a precise finding. The formula failed for a stated reason. That precision is what separates a negative result from a dead end. The N=4 scaling cliff carried the same lesson. Phase 6's failure could have been treated as a dead end. Designing Phase 7 to locate the boundary precisely was the decision that produced a genuinely new empirical finding.

The decision that paid off most in practice was building the telemetry pipeline before any results existed. Writing the benchmark orchestrator and the zone-level logging in Phase 0 felt like overhead when I was trying to get to the MARL experiments. It became necessary the moment Phase 3 started producing collapsing agents. The telemetry was already in place to distinguish collapse from instability and to attribute it specifically to replay buffer contamination. Without

that infrastructure, the Phase 3 collapse rate table would have been a qualitative observation rather than a quantified finding with a mechanistic explanation.

If I were starting again I would maintain five seeds per cell throughout rather than reducing to three. The reduction widened confidence intervals and was a trade against time that I would not make again. I would also build the Phase 5 non-zero-sum structure into the design from the start rather than arriving at it through Phase 4. That is easy to say in retrospect. Phase 4 produced a standalone result I would not discard. But Phase 5 is where the question the project was actually asking finally became testable. Reaching it through Phase 4's negative result added an experimental phase that could instead have been a design decision.

I came into this trying to combine interests I had not previously brought together. I think, on balance, it worked.

A.3 Legal, social, ethical and professional issues

A.3.1 Legal issues

All external libraries used in this project are open-source. FastF1 is released under the MIT licence. PyTorch is released under a BSD-style licence. NumPy and Pandas are released under BSD licences. The Anthropic Claude Haiku API was used under Anthropic's standard commercial terms of service. The FastF1 telemetry data is drawn from publicly available Formula 1 race data intended for non-commercial research use, consistent with the library's intended purpose and licence terms.

A.3.2 Social issues

This project does not interact with human participants and does not collect or process personal data. The research question concerns the behaviour of autonomous software agents in a simulated environment. There are no direct social impacts from the project itself.

The broader research area has social relevance. Autonomous agents with configurable incentive structures are deployed in domains beyond motorsport simulation, including financial trading, logistics optimisation, and competitive resource allocation. The finding that incentive formula design shapes emergent agent behaviour as strongly as algorithm choice is relevant to anyone designing the reward contracts for multi-agent systems in these domains. The project provides controlled evidence that poorly designed cooperative incentives can produce competitive or even harmful agent behaviour, which is a meaningful contribution to the broader conversation about AI alignment in multi-agent settings.

A.3.3 Ethical issues

No human participants were involved at any stage of this project. No consent forms were required. All data generated and used was produced by me and does not involve external parties, which limits the risk of data mishandling or unintended data linkage.

The LLM baseline in Phase 9 uses the Anthropic Claude Haiku API. No personal data was transmitted to the API at any point. All race state information is synthetic and contains no identifying information.

External literature is used as reference material throughout the report. I have been careful to cite sources accurately, to check that referenced works are openly available for academic use, and to confirm that each source is a legitimate published work. One citation was found to be unverifiable during the final review process and was removed.

Running simulations and training agents over extended periods carries a non-trivial energy cost. I was mindful of this throughout and avoided running experiments whose outputs I had no intention of using. The phase-gated structure supported this by requiring a gate to pass before the next phase's compute was committed.

The most direct ethical concern for this type of project is the reliability of the policies it produces. This simulator abstracts away substantial physical complexity. There is no tyre degradation, no pit stop timing, no weather variation, and no driver fatigue. A policy that performs well in this environment may behave very differently under real race conditions, where these factors interact with overtaking decisions in ways the simulator does not capture. If a real driver or team adopted strategies derived from this system without understanding those limitations, they could make overtaking attempts that the simulator rates as favourable but that carry materially higher real-world risk. The Phase 9 result is instructive. A zero-shot language model with no training matched rainbow-lite within two per cent win rate, which suggests the learned strategy may be less general than win rates alone imply. The La Source dominance artefact is a concrete example of a simulator-specific bias that would not transfer to circuits without a structurally straightforward first braking zone. Any real-world application of this system would require explicit validation against the circuits, conditions, and risk tolerances it was not designed to cover.

A.3.4 Professional issues

This project follows the reproducibility standards identified as necessary for trustworthy reinforcement learning comparisons (Henderson et al., 2018; Agarwal et al., 2021). Matched compute budgets, fixed seeds, confidence interval reporting, and behavioural diagnostics that go beyond outcome-only metrics are treated as professional obligations rather than optional additions. The git repository provides a full audit trail of experimental code, configuration changes, and metric outputs. All claims in the main report are traceable to specific metric files or mathematical derivations.

Appendix B

External Material

All materials not developed by the student are listed below.

Table B.1: External materials used in this project

Material	**Version**	**Purpose**	**Licence**
FastF1	v3.x	2023 Belgian Grand Prix telemetry for overtaking zone difficulty calibration	MIT
PyTorch	v2.x	Neural network implementation for all DQN-family variants	BSD
NumPy	v1.x	Numerical computation in simulator and metrics scripts	BSD
Pandas	v2.x	Metrics aggregation and results table generation	BSD
Anthropic Claude Haiku	API (2024)	Zero-shot LLM agent for Phase 9 semantic baseline	Commercial API terms of service

All academic papers cited in this report were accessed through publicly available sources or the University of Leeds library. No software developed for this project relies on proprietary algorithms or datasets. The project codebase is available to the supervisor and assessor via the git repository linked in the project submission.

The related work comparison table from the original report is reproduced in Table B.2 below for reference.

B.1 Overtaking zone calibration figures

The following figures show the FastF1 telemetry analysis used to calibrate overtaking zone difficulty values in Section 2.1. Figure B.1 plots the absolute number of position changes against track distance at the 2023 Belgian Grand Prix. Figure B.2 overlays overtake density on the Spa-Francorchamps circuit layout. Both are derived from publicly available race data accessed via the FastF1 library.

Table B.2: Comparison of related work and relevance to this dissertation

Study	**Domain and simulator fidelity**	**Decision method**	**Uncertainty treatment**	**Evaluation rigour**	**Limitations**	**Direct relevance to thesis**
Heilmeier et al. (2018)	Circuit motorsport with lap-wise race simulation and tyre and fuel effects	Strategy simulation and what-if analysis	Includes overtaking and degradation effects in model dynamics	Practical validation on historical race setting	Not focused on RL benchmark fairness across seeds	Motivates staged simulator design with tyre extension
Heilmeier et al. (2020)	Circuit motorsport strategy support with ANN components	Surrogate modelling and optimisation support	Captures race variability through modelled scenarios	Strong engineering framing and deployability discussion	Limited focus on RL reproducibility protocol	Supports need for interpretable strategy tools
Piccinotti et al. (2021)	Formula 1 strategy identification in planning setting	Online planning methods	Dynamic race context in planning loop	Demonstrates value of planning for strategy choice	Limited DQN-family comparative analysis	Clarifies boundary between planning and RL benchmarking goals
Liu and Fotouhi (2020)	Formula E race strategy with energy constraints	ANN plus MCTS	Multi-stage uncertainty in energy and race conditions	Demonstrates decision quality in time-critical domain	Not designed as controlled RL reproducibility benchmark	Supports tactical decision framing under uncertainty
Liu et al. (2024)	Formula E multi-car strategy learning context	Reinforcement learning based strategy development	Competitive race uncertainty in multi-car setting	Strong domain result emphasis	Less emphasis on seed-controlled statistical protocol	Provides adjacent motorsport RL evidence base
Mnih et al. (2015), van Hasselt et al. (2016), Wang et al. (2016), Hessel et al. (2018)	General RL benchmarks	DQN, Double DQN, Dueling DQN, Rainbow	Algorithmic robustness mechanisms in value learning	Benchmark-oriented algorithm studies	Not motorsport-specific and limited tactical interpretability	Defines core algorithm comparison set for this dissertation
Henderson et al. (2018), Agarwal et al. (2021)	Cross-domain deep RL evaluation	Reproducibility and statistical evaluation methodology	Focus on stochastic outcome variation and seed effects	Strong critique of weak reporting practice	Do not provide motorsport-specific protocol	Directly motivates fixed-seed and interval-based evaluation design
Sankary and Ostfeld (2018)	Robust optimisation in water systems	Evolutionary optimisation with stochastic scenario selection	Explicit stochastic scenario resampling	Robustness-focused evaluation method	Different optimisation paradigm from RL	Supports uncertainty-aware evaluation discipline
This dissertation	F1-inspired competitive simulator with staged complexity	Reproducible DQN-family benchmark with behavioural diagnostics	Controlled stochasticity pathway with tyre-related uncertainty stage	Matched budget and seed-controlled reporting with confidence intervals	Simplified simulator abstraction and staged feature scope	Targets interpretable and statistically grounded method comparison

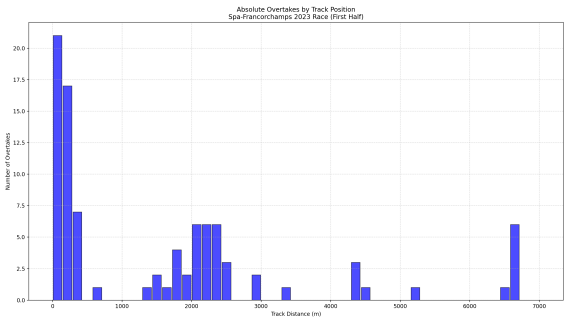

Figure B.1: Absolute overtakes by track distance, 2023 Belgian Grand Prix.

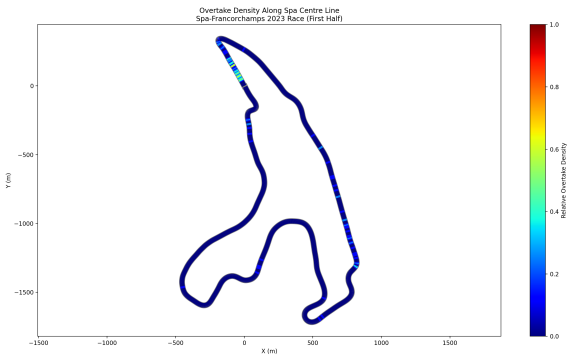

Figure B.2: Overtake density mapped to the Spa-Francorchamps circuit.

Appendix C

Phase 0 and Phase 1 Experimental Validation

This appendix documents the system integrity checks carried out in Phase 0 and the smoke evaluation results collected in Phase 1. Both phases are prerequisites to the Phase 2 DQN-family benchmark. Phase 0 verified that the simulator, reward accounting, stochasticity, and telemetry were correct before any learning claims were made. Phase 1 confirmed a learning signal at low compute cost and established the stochasticity sensitivity profile of the vanilla DQN baseline.

C.1 Phase 0: System Integrity

C.1.1 Objective and scope

Phase 0 contained twelve white-box unit and integration tests grouped into three categories. Group A checked simulator correctness. Group B validated the stochastic probability calculations statistically. Group C verified the telemetry contract. All tests were implemented in `tests/test_phase0_integrity.py` and executed under a fixed random seed with the Spa circuit at low complexity and stochasticity level s0.

C.1.2 Test inventory

Table C.1: Phase 0 integrity test inventory and outcomes

Grp	Test name	What it checks	Result
A	`test_one_decision_per_zone_per_lap`	Each (driver, zone, lap) tuple appears at most once per run in the decision event log	Pass
A	`test_cooldown_enforcement`	Any two sequential attempt events for the same driver are separated by at least 5.0 s of simulated time	Pass
A	`test_terminal_reward_correctness`	`outcome_raw` equals starting position minus final position for every driver	Pass
A	`test_reward_sum_identity`	`reward_component_sum_error` is below 10^{-4} for every decision event	Pass
A	`test_inactive_components_zero`	`pace` and `tyre_pit` components are exactly 0.0 in every event at low complexity	Pass
B	`test_aggression_increases_probability`	Mean success probability across 1000 samples satisfies AGGRESSIVE > NORMAL > CONSERVATIVE at fixed zone and gap	Pass
B	`test_gap_decreases_probability`	Closer gap yields higher mean success probability at 1000 samples	Pass
B	`test_zone_difficulty_scales_probability`	Lower difficulty zone yields higher mean success probability at 1000 samples	Pass
C	`test_jsonl_schema`	Required top-level keys present in every race results JSONL record	Pass
C	`test_decision_event_schema`	Required per-decision keys present including `reward_component_sum_error` and `success_probability`	Pass
C	`test_component_totals_match_events`	Per-driver `reward_component_totals` match the sum of individual decision events for decision-time components	Pass

C.1.3 Defects found and corrected

Two defects were identified during Phase 0 and corrected before any Phase 1 runs were executed.

The first defect was in the base agent heuristic. The `default_policy` function in `src/base_agents.py` reads the gap to the car ahead via `getattr(driver, "gap_to_ahead", None)`. The `DriverState`

class did not initialise this attribute, so the value was always `None`. This collapsed the base agent's overtake attempt probability to approximately 19% at La Source instead of the intended 77%, making the base agent effectively passive. The defect was corrected by setting `driver.gap_to_ahead = gap_distance` in the simulator's `_check_overtakes` method before the agent action call. After correction the base agent attempted overtakes at a frequency consistent with its heuristic policy parameters.

The second defect was a scoping error in `test_one_decision_per_zone_per_lap`. The deduplication set was initially keyed on driver name alone rather than on the (run number, driver name) pair, causing legitimate across-run repetitions to trigger false failures. The test was corrected to scope deduplication within each individual run.

C.1.4 Phase 0 gate outcome

All twelve tests passed. No `reward_component_sum_error` exceeded 10^{-4}. The stochastic calibration ordering assertions held across all 1000-sample replicates. The JSONL schema contained all required keys. Gate G0 was cleared and Phase 1 proceeded.

C.2 Phase 1: Single-Agent Smoke Evaluation

C.2.1 Objective and configuration

Phase 1 confirmed that a learning signal existed before the full Phase 2 benchmark matrix was run. All five smoke runs used a vanilla DQN agent trained for 200 episodes against a single heuristic base agent at low complexity on the Spa circuit. The epsilon decay was set to 0.995, producing $\varepsilon \approx 0.37$ by episode 200. Each run was evaluated over 20 races per seed across three evaluation seeds (101, 202, 303). The three s0 runs used independent training seeds (101, 202, 303). The s1 and s2 runs used training seed 101 only.

The stochasticity levels used Gaussian noise on the overtake success probability. s0 used noise standard deviation 0.0 (deterministic). s1 used 0.02. s2 used 0.05.

C.2.2 s0 results: three training seeds

Table C.2: Phase 1 s0 smoke results across three training seeds (60 eval races per seed set)

Train seed	Win rate	CI95 low	CI95 high	Position delta	Seed var.	DNF rate
101	0.783	0.678	0.888	0.567	0.0033	0.0
202	0.767	0.659	0.875	0.533	0.0058	0.0
303	0.783	0.678	0.888	0.567	0.0033	0.0

All three seeds exceeded the Gate G1 criterion of win rate above 0.50 with CI95 lower bound above 0.50. Starting position exposure was perfectly balanced at 30 races from each of positions 1 and 2 in every run. No DNFs occurred. Gate G1 was cleared.

The three seeds produced closely grouped results, with seed variance between 0.003 and 0.006. This indicated convergence to a similar policy attractor across independent training runs.

C.2.3 Risk behaviour at s0

Table C.3: Risk level counts and zone-level attempt rates at s0 (train seed 101, 60 eval races)

Zone	Difficulty	Decisions	Attempt rate	Success rate	Avg reward
La Source	0.2	30	1.00	0.867	+0.127
Raidillon	0.9	28	1.00	0.143	−0.021
Les Combes	0.7	46	0.870	0.300	+0.013
Bruxelles	0.7	13	1.00	0.231	+0.015

Risk level counts at s0 were CONSERVATIVE 60, NORMAL 42, AGGRESSIVE 10. The agent showed a strong preference for cheaper-failure strategies at s0. Only four of nine configured zones appeared in any decision event. Zones at greater track distances from the start were not reached in sufficient proximity situations to trigger decisions within the two-car five-lap race format.

The agent attempted at Raidillon (difficulty 0.9) 100% of the time despite a 14% success rate and a net-negative average reward. This indicated that the policy had not learned to discriminate HOLD decisions at high-difficulty zones within 200 training episodes.

C.2.4 Stochasticity degradation: s0, s1, and s2

Table C.4: Phase 1 stochasticity comparison (train seed 101, 60 eval races per level)

Level	Win rate	CI95 low	CI95 high	Tactical raw total	Outcome raw total	Seed var.
s0	0.783	0.678	0.888	−11.0	+17.0	0.0033
s1	0.783	0.678	0.888	−17.5	+17.0	0.0033
s2	0.750	0.640	0.860	−27.5	+15.0	0.0100

Table C.5: Risk level counts across stochasticity levels (train seed 101, 60 eval races per level)

Level	CONSERVATIVE	NORMAL	AGGRESSIVE	Decisions	Attempt rate/run
s0	60	42	10	118	1.87
s1	19	26	48	110	1.55
s2	18	30	49	97	1.62

C.3 Phase 1 findings and implications

C.3.1 Win rate floor: La Source structural dominance

The win rate was flat from s0 to s1 (0.783 in both cases). This was not a sign of robustness. The dominant overtaking zone at La Source (difficulty 0.2) produced an 87–90% success rate in all three levels with 100% attempt rate. Adding a noise standard deviation of 0.02 to a near-certain probability event is insufficient to flip race outcomes at La Source. The s1 noise level fell below

the La Source erosion threshold, making the s0-to-s1 comparison uninformative about true policy robustness.

The tactically informative metric was the win rate from position-two starts. Stripping out the structural position-one advantage, the agent won approximately 57% of P2-start races at both s0 and s1, falling to 50% at s2. This 7 percentage point reduction at s2 was the genuine sensitivity signal.

C.3.2 AGGRESSIVE flip as a DQN overestimation indicator

The risk level distribution reversed completely between s0 and s1. At s0 the agent used CONSERVATIVE in 53% of attempts. At s1 and s2 it used AGGRESSIVE in 52–51% of attempts. This reversal persisted across stochasticity levels once noise was introduced.

The root cause was Q-value overestimation under high-variance rewards. Vanilla DQN uses a max operator in the Bellman update, which preferentially propagates optimistic samples. Under noisy outcomes, AGGRESSIVE occasionally produced high-reward events. These upward noise samples inflated Q-values for AGGRESSIVE in training, biasing the policy toward the highest-penalty failure strategy at evaluation. The consequence appeared directly in the tactical component, which worsened from -11.0 at s0 to -27.5 at s2, a $2.5\times$ increase despite fewer total decisions.

This overestimation pattern under noise is the documented failure mode that motivated Double DQN and is empirically visible here. It provides direct motivation for the Phase 2 algorithm comparison.

C.3.3 Zone discrimination collapse at s2

At s0 the agent applied a partial HOLD policy at Les Combes (87% attempt rate). At s1 this discrimination strengthened to 73% attempt rate, and the Raidillon HOLD rate improved to 90%. At s2 both zones reverted to 100% attempt rate. The learned discrimination did not persist beyond moderate noise. This indicated that the Q-value signal was too weak at 200 training episodes to maintain zone-specific HOLD behaviour when outcome variance exceeded the signal from the penalty gradient.

C.3.4 Seed variance as a robustness indicator

Seed variance was stable at 0.0033 across s0 and s1, then tripled to 0.010 at s2. This was the first measurable instability signal and the entry point for the RQ1 robustness analysis. Policy sensitivity to training seed increased at s2, confirming that higher noise was beginning to disrupt convergence reliability even within the 200-episode budget.

Table C.6: Phase 1 gate G1 checklist

Criterion	**Outcome**
At least one of three s0 seeds shows win rate above 0.50	Met (all three)
All CI95 lower bounds above 0.50 at s0	Met (lowest: 0.659)
Fairness violations equal zero (start position imbalance)	Met (ratio 0.0)
s1 and s2 show interpretable degradation direction	Met (see note)

C.3.5 Phase 1 gate summary

The degradation direction was interpretable but structurally attenuated. The headline win rate was flat at s1 due to La Source dominance, and only the s2 comparison showed a reduction. The behavioural degradation signal (AGGRESSIVE flip, zone discrimination collapse, rising tactical penalty, increased seed variance) was present and mechanistically consistent across s1 and s2. Gate G1 was cleared and Phase 2 proceeded with this understanding recorded.

Appendix D

DQN variant formulations and training specifics

This appendix gives the formal equations for each DQN variant, the network and replay specifications, the exploration and optimisation hyperparameters, and the protocol by which those hyperparameters were selected. The conceptual descriptions in Section 3.2.1 summarise what each formulation does. This appendix gives the corresponding equations for readers who want the formal definitions alongside.

D.1 Variant TD targets

Every variant trains its Q-network by adjusting its predictions toward a target value y. The target is derived from the observed reward plus a discounted estimate of future value. The target is what differs between variants, and each variant's choice of target corresponds to the mechanism described in the main text.

Vanilla DQN. The vanilla TD target takes the maximum Q-value over next-state actions under the target network,

$$y = r + \gamma \max_{a'} Q_{\text{target}}(s', a')$$

where r is the reward received after the action, γ is the discount factor controlling how much future rewards are weighted relative to immediate ones, s' is the next state, and a' is the action taken there (Mnih et al., 2015).

Double DQN. Double DQN splits action selection from action evaluation, using the online network to pick the action and the target network to grade it,

$$y = r + \gamma Q_{\text{target}}\left(s',\, \arg\max_{a'} Q_{\text{online}}(s', a')\right)$$

This prevents noise in the online network from feeding back into inflated target values (van Hasselt et al., 2016).

Dueling DQN. The dueling architecture reconstructs the Q-value from a state-value stream $V(s)$ and an action-advantage stream $A(s, a)$,

$$Q(s, a) = V(s) + \left(A(s, a) - \frac{1}{|\mathcal{A}|} \sum_{a'} A(s, a')\right)$$

Subtracting the mean advantage ensures V and A are uniquely identified from their sum (Wang et al., 2016).

Rainbow-lite. Rainbow-lite combines double targeting, the dueling architecture, and two fur-

ther mechanisms. Multi-step returns use three consecutive rewards before bootstrapping,

$$y = r_t + \gamma r_{t+1} + \gamma^2 r_{t+2} + \gamma^3 \max_{a'} Q_{\text{target}}(s_{t+3}, a')$$

which propagates reward signal faster than single-step targets (Hessel et al., 2018). Prioritised experience replay samples transitions with probability proportional to $|TD_{\text{error}}|^{\alpha_{\text{PER}}}$, where $|TD_{\text{error}}|$ is the magnitude of the temporal-difference prediction error and the priority exponent $\alpha_{\text{PER}} = 0.6$ (distinct from the reward-sharing α used elsewhere in the report). Importance-sampling weights are annealed from $\beta = 0.4$ to 1.0 over 100,000 steps to correct the introduced bias (Schaul et al., 2016).

D.2 Network and replay specifications

All variants use a two-hidden-layer feedforward neural network with rectified linear unit (ReLU) activations and a hidden layer width of 512 units. Vanilla, Double, and Dueling variants use a uniform circular replay buffer of capacity 150,000 transitions, with samples drawn uniformly at random. Rainbow-lite uses the same buffer capacity with prioritised sampling as described above. The target network is a periodically refreshed copy of the online network, synchronised by hard copy every 1,000 gradient steps.

D.3 Exploration and optimisation schedule

Epsilon starts at 1.0 (fully random action selection) and decays multiplicatively by a factor of 0.995 per episode toward a minimum floor. At the 500-episode Phase 2 budget this leaves $\epsilon \approx 0.08$, so the agent selects a random action roughly 8 per cent of the time at the end of training and exploits its learned policy the rest of the time. The Adam optimiser is used with a learning rate of 7×10^{-4}, and batch size is 64 throughout.

D.4 Hyperparameter tuning protocol

The shared hyperparameters were determined using Autoresearch (Karpathy, 2024), a tuning tool that iteratively proposes and evaluates single-parameter changes against a scored objective. Thirty iterations were run, each scored across three seeds with fifty training runs per seed, before the configuration was committed. Tuning ran against the Phase 2 single-agent contract, so the hyperparameters reflect single-agent stability rather than multi-agent dynamics. This keeps the DQN-variant comparisons fair within Phase 2 and avoids tuning bias that could favour any particular variant in later phases.

Appendix E

DQN Training Loop

The figure below shows the training loop shared across all four DQN-family variants. Each episode the agent selects an action via epsilon-greedy policy, stores the transition in the replay buffer, samples a minibatch, computes the TD target, and updates network weights. The variant-specific logic is confined entirely to the TD target computation step. All other steps are identical across vanilla, double, dueling, and rainbow-lite.

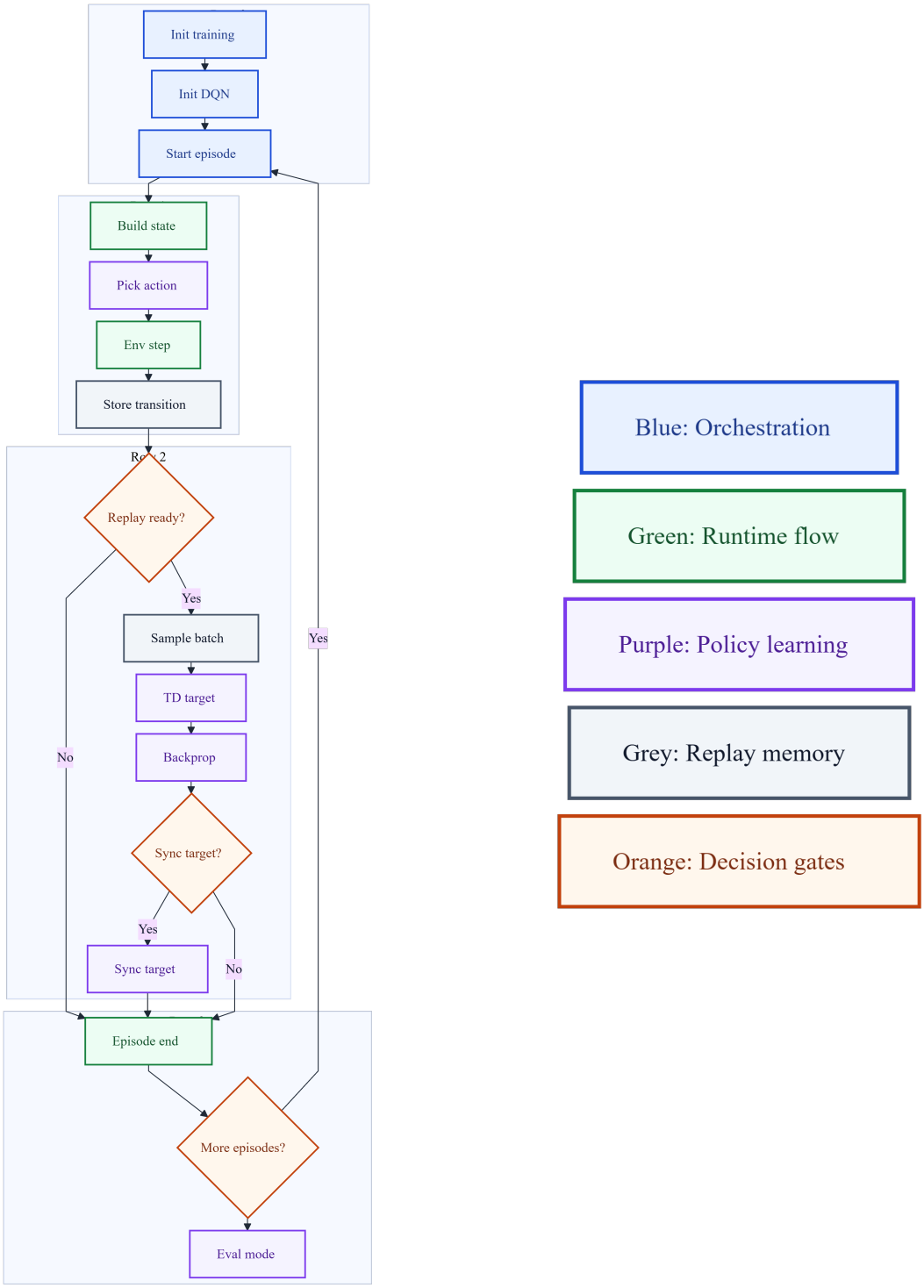

Figure E.1: DQN training loop shared across all four variants. Variant-specific logic is confined to the TD target computation step.

Appendix F

System Class Diagram

F.1 Per-module responsibilities

The simulator is structured as a set of well-separated Python modules, each with a single responsibility. The entry point for a single train-then-evaluate trial resolves the active complexity profile from configuration and passes it to the simulator engine. A companion orchestration script sequences calls across the full algorithm comparison matrix.

The simulator engine owns the tick-by-tick race loop, maintaining race state that tracks all driver positions, lap counts, and time gaps at each simulation step. At each overtaking zone decision point, the engine constructs the observation vector for the approaching driver and queries the driver's agent for an action. The overtake resolution outcome is sampled from the probability function described in Section 2.2.1 and logged to telemetry. At the end of each episode the engine computes terminal reward components and triggers epsilon decay for each DQN-type driver.

All agents implement a common interface providing an action-selection method that the simulator calls uniformly. This abstraction means the simulator does not need to distinguish between algorithm variants at the engine level. It receives a structured action containing a risk level and an overtake attempt flag regardless of which variant is active.

A real-time visualisation mode renders the circuit with agent positions, lap count, and time gap annotations, used during development to verify zone sequencing and reward log consistency. It is disabled for benchmark runs.

F.2 UML class diagram

The figure below shows the full UML class diagram for the simulator core. All DQN-family agent types inherit from the abstract `BaseAgent` interface. `RaceSimulator` owns runtime state through `RaceState` and delegates observation construction to `DriverFeedback`. The `DQNAgent` class is parameterised via `dqn_params.algo` in `config.json`, selecting the active variant without changing any interface contract.

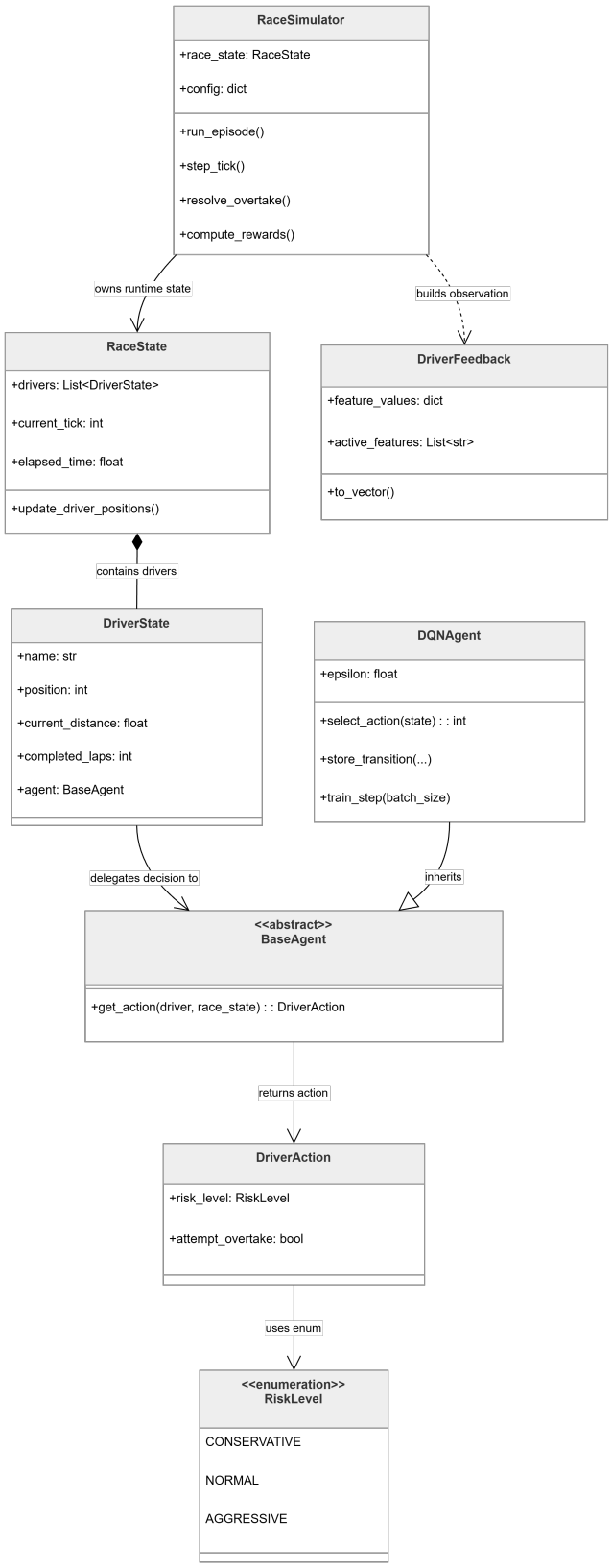

Figure F.1: UML class diagram of the F1 simulator core.

Appendix G

Simulator specification details

This appendix gives the full specifications of simulator elements summarised in Chapters 2 and 3.

G.1 Stochasticity levels

The three stochasticity levels differ only in the standard deviation σ of the Gaussian noise term ϵ applied to the overtake success probability (Section 2.2.1). All other scaling factors in the probability function are held constant across levels.

- **s0:** $\sigma = 0$. The success probability is fixed for a given state, though outcomes still resolve through Bernoulli sampling. This is the primary comparison track.
- **s1:** $\sigma = 0.02$. Introduces modest outcome variance while preserving zone difficulty ordering.
- **s2:** $\sigma = 0.05$. Sufficient to flip borderline outcomes while still preserving zone ordering in expectation.

G.2 Base Agent specification

The Base Agent is a gap-aware heuristic policy used as the fixed non-learning comparator. Its decision logic is as follows.

At each simulation step the agent only considers overtaking when (a) a car ahead is within 100 m and (b) an overtaking zone is imminent. If either condition fails, the agent holds position.

Within the decision window the agent identifies the upcoming overtaking zone, reads its difficulty and the gap to the car ahead, and constructs a soft preference distribution over the three risk levels. Easier zones and closer gaps increase the weight on AGGRESSIVE. Harder zones and larger gaps increase the weight on CONSERVATIVE. A risk level is sampled from this distribution, and an overtake attempt is then made with a probability that depends on the sampled risk level and the gap. The agent's policy is deterministic in its structure but stochastic in its sampling, and does not update across episodes.

G.3 Run reproducibility through telemetry

Every run writes its executed configuration into the output metrics file. The embedded snapshot contains the active `config.json` state, the training and evaluation budgets, the seed assignments

for training and evaluation runs, and the complexity and stochasticity profile. Downstream analysis reads algorithm, hyperparameter, and scenario fields from the telemetry output rather than from the config file on disk, so any mismatch between what was intended to run and what actually ran shows up during analysis rather than silently. Any experiment can be reproduced from the telemetry record alone without recovering the repository state at the time the run was performed.

Appendix H

Benchmark infrastructure specification

This appendix expands the benchmark infrastructure summary in Chapter 3.

H.1 Single-trial runner

The single-trial runner executes one train-then-evaluate trial for a given algorithm variant. It accepts arguments for the training budget, evaluation budget, training seed, evaluation seeds, complexity profile, stochasticity level, and output path. The script first runs training episodes with the agent in exploration mode, then freezes the policy and runs evaluation episodes with exploration disabled. At the end of the run it writes a structured metrics file containing the win rate, objective score, fairness diagnostics, and full behavioural diagnostics including the per-zone attempt and success breakdown and risk distribution.

H.2 Benchmark matrix orchestrator

The benchmark orchestrator sequences single-trial runs across the full comparison matrix, which contains every combination of algorithm, seed, and stochasticity level. Each combination runs as an independent trial. The orchestrator checks fairness against a declared contract specifying the training and evaluation budgets, seed sets, complexity profile, and stochasticity level that all algorithms must share. If any violation is detected, the orchestrator aborts. On completion it writes per-algorithm statistics with 95% CIs and a summary table for rapid inspection.

A companion aggregation script collects the per-seed metrics files produced by Phase 2 and computes means, standard deviations, and 95% CIs across seeds. The Phase 2 algorithm comparison in Chapter 4 is derived from this script, while later phases use the MARL evaluation runner described in Section 2.3.

H.3 Telemetry schema

Every race produces a JSON Lines (JSONL) entry containing the full per-driver result. Each entry includes the driver name, final race position, total reward, per-component reward totals, stochasticity level, complexity profile, and a `decision_events` array. Each decision event records the zone identifier, lap number, action label, attempt flag, success flag, success probability, reward received, individual reward component values, and a `reward_component_sum_error` field that verifies the arithmetic consistency of the reward decomposition. The telemetry schema is validated in Phase 0 Group C tests (Appendix C).

Appendix I

Phase 3 crossplay bias arithmetic

This appendix gives the detailed arithmetic behind the crossplay bias correction summarised in Chapter 4.

A crossplay protocol ran for rainbow-lite at s2 across three seeds, using the self-play and swapped conditions described in Chapter 3. The self-play condition revealed an 11 percentage point structural bias against the A1 evaluation slot, meaning identical policies evaluated in A1 won about 0.445 of races against their copies in A2 rather than the expected 0.500.

Two diagnostic follow-up seeds (404 and 505) were added to the original three to test whether the observed A1 deficit across rainbow-lite seeds reflected systematic policy asymmetry or an evaluation artefact. Seed 505 produced an A1 win rate of 0.560 at s2, breaking the apparent ceiling established by the original three seeds and indicating that the A2 edge was not a property of the trained policies themselves.

Applying the 11 percentage point bias correction to raw A1 win rates places four of five rainbow-lite seeds at or above parity. Only seed 101 retains a genuine A2 edge after correction. The apparent A2 dominance at s2 therefore largely dissolves under correction, supporting the evaluation-bias interpretation as opposed to a systematic policy effect. Collapse classification is unaffected by this correction because collapse is determined by attempt rates and reward trajectories rather than by the win-rate comparison between slots.

Appendix J

Phase 4 secondary patterns

This appendix expands the two secondary patterns summarised in Chapter 4's Phase 4 discussion.

J.1 Non-monotonic zone convention formation

Non-degenerate zone differentiation across the Phase 4 α sweep peaks at $\alpha = 0.25$ and declines thereafter. Partial alignment initially promotes differentiation as agents coordinate on complementary zones, with each agent specialising in a subset of zones rather than both contesting the same high-value locations. As alignment strengthens past this peak, the convention erases. Both agents converge to a risk-averse policy under shared cooperative reward, which flattens zone differentiation back toward zero even before collapse sets in. Convention formation and threshold stability therefore sit at different α values. The zone-convention peak at $\alpha = 0.25$ is well below the collapse threshold between $\alpha = 0.5$ and $\alpha = 0.75$.

J.2 Stochasticity role inversion

Stochasticity plays different roles on either side of the collapse threshold. At $\alpha \leq 0.5$ noise is stabilising, breaking potential lock-in patterns before they consolidate. Agents that might otherwise settle into a premature equilibrium are jolted into further exploration by outcome variance. At $\alpha \geq 0.75$ the same noise accelerates collapse, because once the cooperative reward term dominates, any early failed attempt under high noise reinforces the passive equilibrium faster than under deterministic outcomes.

Noise also weakens zone conventions themselves. The $\alpha = 0.25$ zone differentiation peak attenuates sharply under increasing stochasticity, with the clearest conventions visible at s0 and the weakest at s2. This pattern reinforces the main-text finding that low-α competitive conventions emerge through precise best-response dynamics, which noise disrupts, while high-α passive equilibria are reached through reward-gradient dominance, which noise reinforces.

Appendix K

MARL and phase extensions

This appendix gives the full specification of codebase extensions made for Phases 3 to 10, summarised in Section 3.3.2.

K.1 Phase 3: Concurrent learners

Phase 3 required the codebase to support two concurrent DQN learners. The complexity profile layer was extended with a two-DQN competitor selector that initialises both agents from the same hyperparameter block, each with separate replay buffers, network parameters, and epsilon schedules. The simulator engine required no changes, as it already triggers a training step on every DQN-type driver at the end of each episode.

K.2 Phases 4 to 10: Reward, team, curriculum, and agent extensions

Phases 4 to 10 were controlled through configuration with no changes to agent learning code. The reward calculation was extended with an α blending step, where setting α to zero reproduces the original reward contract exactly. A team-grouping selector was added to assign agents to named teams for α mixing. A curriculum scheduler was implemented as a thin wrapper around the training loop, reading the target α and episode boundaries from config and hooking into the existing episode-end callback. A difference reward mode computes the mean-field approximation described in Section 2.3.13. The LLM agent implements the same common interface as all other agents, with its prompt programmatically constructed from simulator state and track configuration around a fixed hand-authored system template (Section 2.3.12). The agent queries Claude Haiku, a language model API provided by Anthropic, once per lap and outputs a structured zone plan. On parse failure it retains the previous lap's plan or installs a default plan, which still allows attempts at easy zones.